

Bivariate Manifold-Adaptive Weights Empirical Cubature Method

Construction, Graph-Regularized Pruning, and Weight Regression

Joaquín A. Hernández Ortega

May 2026

Abstract

See shorter version in [02_MAW_ECM.pdf](#).

This document develops, from first principles, the formulation implemented in `MAW_ECM_genV1.m`. The method is a manifold-adaptive extension of the Empirical Cubature Method (ECM), constructed for nonlinear manifold reduced-order models in which the relevant reduced states lie on a low-dimensional latent manifold.

The formulation originates from the idea of combining a common reduced integration support with manifold-dependent cubature weights. The resulting method may be interpreted as a manifold counterpart of subspace-adaptive cubature constructions such as SAW-ECM, in which the support is shared across a family of reduced representations while the weights are allowed to vary. Accordingly, the central object in MAW-ECM is not a single cubature vector, but a family of adaptive weights

$$\mathbf{w} = \mathbf{w}(\mathbf{q}_M),$$

where $\mathbf{q}_M$ denotes the latent coordinates of the nonlinear manifold.

At the conceptual level, the method may be viewed as a manifold-adaptive sparsification problem: one seeks the smallest common support capable of reproducing, at every sampled manifold state, a collection of local exactness constraints associated with reduced residuals and outputs. However, the direct solution of this global support-selection problem is computationally intractable because it combines combinatorial sparsity, positivity, local exactness, and manifold regularity constraints in a single coupled formulation.

The implemented MAW-ECM construction therefore does not solve the original sparsification problem directly. Instead, it proceeds through a feasible pruning strategy starting from an already admissible fixed-weight ECM rule. The initial ECM support is progressively reduced by removing candidate integration points one at a time and redistributing the remaining weights over the manifold while preserving the local exactness conditions. In this sense, sparsification defines the ideal mathematical objective, whereas pruning defines the practical algorithmic route.

The redistribution process operates in two regimes. First, an inexpensive least-change stage attempts to eliminate points without explicit positivity enforcement. Once this becomes impossible, the method switches to a graph-regularized active-set redistribution stage in which positivity is enforced while neighboring manifold states are coupled through a graph Laplacian. This graph coupling is introduced to moderate the loss of regularity caused by local active-set transitions in the adaptive weights.

The retained integration points are common to all sampled manifold states, whereas the associated weights vary over the latent manifold. Consequently, the method lies between a fully global ECM rule, where both support and weights are fixed, and a fully local cubature strategy, where both support and weights would be recomputed independently at each state.

The present implementation is specialized to a bivariate latent manifold, in which the sampled states lie on a structured two-dimensional grid. This assumption is used primarily in the construction of the graph regularization operator and in the visualization of the adaptive weight fields. The algebraic structure of MAW-ECM itself is more general and extends naturally to latent manifolds of dimension $r_M > 2$, provided that a suitable neighborhood graph or discrete manifold connectivity is available.

Contents

1	Purpose and scope of the document	4
2	Relation with classical ECM	4
3	Introduction: from adaptive cubature supports to manifold-adaptive weights	5
3.1	Roots in SAW–ECM: common support with adaptive weights	5
3.2	The full manifold-adaptive sparsification problem	6
3.3	Why the full formulation is not used directly	7
3.4	Restriction to an initial ECM support	7
3.5	CECM influence: pruning instead of direct sparsification	8
3.6	From the full global step to the implemented two-regime algorithm	8
3.7	Role of positivity and active sets	9
3.8	Current formulation	9
4	Training manifold and local exactness systems	10
5	Fixed support and adaptive weights	10
6	Bivariate graph structure	11
7	Global MAW–ECM algorithm	12
8	Interpretation of the algorithm	14
9	Specific and general components	14
9.1	General MAW–ECM components	14
9.2	Components specific to the two-variable implementation	15
9.3	Extension to higher-dimensional latent manifolds	15
10	Construction of the local exactness systems	15
10.1	Fixed scaffold	16
10.2	Weighted projector	16
10.3	Local basis at a sampled manifold state	17
10.4	Initialization of the adaptive weights	18
10.5	Function-level interpretation	18
10.6	Specificity and possible generalizations	19
11	Graph Laplacian on the sampled latent manifold	20
11.1	Structured bivariate graph	20
11.2	Finite-element-like graph stiffness	21
11.3	Incremental smoothing	21
11.4	Stacked-vector form	22
11.5	Boundary-weighted graph regularization	22
11.6	Specificity and extension to higher dimensions	23
11.7	Function-level interpretation	23
12	Pruning strategy and candidate elimination	23
12.1	Active and eliminated sets	24
12.2	Candidate ranking by aggregate weight	24
12.3	Single-point elimination trial	25
12.4	Stopping criterion	25
12.5	Two-stage pruning logic	25

12.6	Pseudo-algorithm for the pruning loop	26
12.7	Interpretation	27
12.8	Function-level interpretation	28
13	Stage 1: no-enforcement least-change redistribution	28
13.1	Local least-change redistribution	29
13.2	Reduced local system after elimination	29
13.3	Closed-form least-change update	30
13.4	Interpretation as a minimum redistribution	30
13.5	Why smoothness is usually better in this stage	31
13.6	Acceptance and rejection of candidates	31
13.7	Pseudo-algorithm	31
13.8	Function-level interpretation	32
14	Stage 2: local positivity enforcement by active sets	33
14.1	Local constrained redistribution problem	33
14.2	Geometric interpretation	34
14.3	Active-set philosophy	34
14.4	Reduced local variables	34
14.5	Iterative evolution of the active sets	35
14.6	Node-dependent active sets	35
14.7	Origin of nonsmoothness	35
14.8	Acceptance and rejection	36
14.9	Pseudo-algorithm	36
14.10	Interpretation	37
14.11	Function-level interpretation	38
15	Stage 3: graph-regularized positivity enforcement	38
15.1	Global unknown and constraints	38
15.2	Incremental graph-regularized objective	39
15.3	Stacked quadratic form	39
15.4	Node-dependent active sets	40
15.5	Nullspace parametrization	40
15.6	Active-set iterations	41
15.7	Why graph coupling helps	41
15.8	Pseudo-algorithm	42
15.9	Function-level interpretation	42
16	Regression of the adaptive weight fields	43
16.1	Weight fields over the latent manifold	43
16.2	Why regression is necessary	43
16.3	Choice of radial basis functions	44
16.4	RBF approximation of one weight field	44
16.5	Structured subsampling of RBF centers	45
16.6	Matrix form of the regression problem	45
16.7	Differentiability	46
16.8	Influence of graph regularization	46
16.9	Online evaluation	47
16.10	Pseudo-algorithm	47
16.11	Function-level interpretation	48

17 A physical interpretation of the pruning process	48
17.1 A suspended bridge under many wind conditions	48
17.2 The frozen-force philosophy of standard ECM	49
17.3 The central idea of MAW–ECM	49
17.4 Pruning as cutting cables	50
17.5 Slacking cables and active sets	50
17.6 Regularization as coherent structural behavior	51
17.7 Final picture	51

1 Purpose and scope of the document

The goal of this note is to fix the mathematical and algorithmic structure of the manifold-adaptive weights ECM, abbreviated here as MAW–ECM. The document is not intended to be inserted directly into a paper. Its purpose is instead to make explicit the ideas implemented in the MATLAB routine

MAW_ECM_genV1.m,

and in the auxiliary routines that it invokes.

The central question addressed by the method is the following. Suppose that a fixed ECM rule has already selected a set of integration points

$$\mathcal{Z}_0 \subset \mathcal{Z}_{\text{FE}} := \{1, \dots, n_{\text{ip}}\},$$

with associated positive weights $\mathbf{w}_0$. This rule is globally valid over the training manifold, but it may still contain more points than appear necessary from the viewpoint of the latent dimension. Can the support be reduced further if the weights are allowed to vary with the manifold coordinate?

MAW–ECM answers this question by replacing a single fixed cubature vector by a matrix of adaptive weights,

$$\mathbf{W} = \begin{bmatrix} | & | & \dots & | \\ \mathbf{w}^{(1)} & \mathbf{w}^{(2)} & \dots & \mathbf{w}^{(N_{\text{M}})} \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{n_{\text{c}} \times N_{\text{M}}}.$$

Here $n_{\text{c}} = |\mathcal{Z}_0|$ is the number of current candidate points and N_{M} is the number of sampled manifold states. The j -th column $\mathbf{w}^{(j)}$ contains the cubature weights associated with the latent sample $\mathbf{q}_{\text{M}}^{(j)}$.

The support is pruned globally: removing one integration point means removing one row of $\mathbf{W}$ for all sampled states. The weights, however, are adapted locally in the manifold:

$$\mathbf{w}^{(j)} \equiv \mathbf{w}(\mathbf{q}_{\text{M}}^{(j)}).$$

The final online approximation is obtained by fitting a regression model to the columns of $\mathbf{W}$, so that the weights can be evaluated at a new latent coordinate $\mathbf{q}_{\text{M}}$.

2 Relation with classical ECM

Let $\mathcal{Z}_{\text{FE}} = \{1, \dots, n_{\text{ip}}\}$ denote the full set of finite element integration points and let $\mathbf{w}^{\text{FE}} \in \mathbb{R}^{n_{\text{ip}}}$ collect the corresponding finite element quadrature weights. For a state-dependent scalar integrand $f_i(\mathbf{q}_{\text{M}})$ evaluated at integration point i , the full quadrature approximation reads

$$I(\mathbf{q}_{\text{M}}) = \sum_{i=1}^{n_{\text{ip}}} f_i(\mathbf{q}_{\text{M}}) w_i^{\text{FE}}.$$

For several scalar integrands collected in a matrix

$$\mathbf{A}(\mathbf{q}_M) \in \mathbb{R}^{n_{\text{ip}} \times m},$$

the full integral vector is

$$\mathbf{b}(\mathbf{q}_M) = \mathbf{A}(\mathbf{q}_M)^T \mathbf{w}^{\text{FE}}.$$

Classical ECM constructs a fixed subset $\mathcal{Z}_{\text{red}} \subset \mathcal{Z}_{\text{FE}}$ and a fixed vector of positive weights $\mathbf{w}_{\text{red}}$ such that

$$\mathbf{A}_{\mathcal{Z}_{\text{red}}}(\mathbf{q}_{Mj})^T \mathbf{w}_{\text{red}} \approx \mathbf{A}(\mathbf{q}_{Mj})^T \mathbf{w}^{\text{FE}}, \quad j = 1, \dots, N_M.$$

The same weights must be valid for all sampled states. Consequently, the fixed rule must represent the union of all relevant integrand configurations over the training manifold. This global requirement is precisely what limits the compressibility of a fixed-weight cubature formula.

MAW-ECM modifies only one aspect of this construction. The support remains fixed after the offline stage, but the weights are allowed to vary with the latent state:

$$\mathbf{A}_{\mathcal{Z}_{\text{red}}}(\mathbf{q}_M)^T \mathbf{w}_{\text{red}}(\mathbf{q}_M) \approx \mathbf{A}(\mathbf{q}_M)^T \mathbf{w}^{\text{FE}}.$$

Thus, the method does not abandon the ECM philosophy. It keeps the idea of a sparse positive cubature rule defined on selected finite element integration points. The difference is that the cubature weights become functions on the reduced manifold.

3 Introduction: from adaptive cubature supports to manifold-adaptive weights

The Manifold-Adaptive Weights Empirical Cubature Method (MAW-ECM) was not conceived as a direct regression strategy for cubature weights, nor as a purely heuristic modification of a standard ECM rule. Its formulation emerged from the attempt to reconcile three requirements that are individually natural but difficult to satisfy simultaneously:

- (i) the cubature rule should use a common reduced set of integration points over a family of reduced states;
- (ii) the cubature weights should be allowed to adapt to the position on a nonlinear manifold;
- (iii) the resulting weight fields should be sufficiently regular to be used in an online hyper-reduced model.

The first requirement is inherited from subspace-adaptive cubature ideas, the second from the manifold character of the reduced model, and the third from the fact that the adaptive weights are not merely postprocessed quantities: they enter the online evaluation of residuals, outputs, and, in nonlinear settings, their consistent linearizations.

3.1 Roots in SAW-ECM: common support with adaptive weights

The first conceptual root of MAW-ECM is the Subspace Adaptive Weights Empirical Cubature Method (SAW-ECM). In SAW-ECM, one considers several reduced subspaces, indexed by $s = 1, \dots, n_s$, and associates with each of them a cubature weight vector

$$\boldsymbol{\zeta}^{(s)} \in \mathbb{R}^{n_{\text{ip}}}.$$

The key point is that sparsity is not imposed independently on each $\boldsymbol{\zeta}^{(s)}$. Instead, sparsity is imposed on the union of their supports. At the formal level, this can be expressed through an objective involving the zero-norm of the aggregate weight vector,

$$\left\| \sum_{s=1}^{n_s} \boldsymbol{\zeta}^{(s)} \right\|_0.$$

Thus, all subspaces share a common set of selected integration points, while the weights are allowed to depend on the subspace.

This idea is central for MAW-ECM. The method keeps the same distinction:

common support versus state-dependent weights.

What changes is the indexing set. SAW-ECM adapts the weights over a finite collection of reduced subspaces. MAW-ECM adapts the weights over a sampled nonlinear manifold of latent states.

Let

$$\mathcal{M}_{\text{tr}} = \{\mathbf{q}_{\text{M}}^{(j)}\}_{j=1}^{N_{\text{M}}}$$

be the sampled training manifold. At each sampled state $\mathbf{q}_{\text{M}}^{(j)}$, one introduces a cubature weight vector

$$\boldsymbol{\zeta}^{(j)} \in \mathbb{R}^{n_{\text{ip}}}.$$

The direct manifold analogue of the SAW-ECM support-union objective is then

$$\left\| \sum_{j=1}^{N_{\text{M}}} \boldsymbol{\zeta}^{(j)} \right\|_0.$$

The support of this aggregate vector identifies the integration points that are used somewhere on the manifold. Since the weights are nonnegative, the support of the sum is precisely the union of the supports of the individual rules.

3.2 The full manifold-adaptive sparsification problem

For each sampled manifold state $\mathbf{q}_{\text{M}}^{(j)}$, let

$$\mathbf{U}_j \in \mathbb{R}^{n_{\text{ip}} \times r_j}, \quad \mathbf{b}_j \in \mathbb{R}^{r_j},$$

denote the local exactness operator and the corresponding vector of exact integrals. The constraints

$$\mathbf{U}_j^T \boldsymbol{\zeta}^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_{\text{M}},$$

represent the quantities that the cubature rule must reproduce at each manifold point. In a manifold HROM, these quantities may include tangent-space projected residual contributions, output quantities such as reactions or homogenized stresses, and the volume constraint. The latter prevents the degenerate zero-weight solution and preserves the total measure represented by the quadrature rule.

A first formal statement of the manifold-adaptive sparsification problem is therefore

$$\begin{aligned} \min_{\boldsymbol{\zeta}^{(1)}, \dots, \boldsymbol{\zeta}^{(N_{\text{M}})}} \quad & \left\| \sum_{j=1}^{N_{\text{M}}} \boldsymbol{\zeta}^{(j)} \right\|_0 \\ \text{subject to} \quad & \mathbf{U}_j^T \boldsymbol{\zeta}^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_{\text{M}}, \\ & \boldsymbol{\zeta}^{(j)} \geq 0, \quad j = 1, \dots, N_{\text{M}}. \end{aligned}$$

This is the original, idealized MAW-ECM problem. It is the exact analogue of the SAW-ECM sparsification principle, but with the discrete subspace index replaced by a sampled manifold index.

However, this formulation is still incomplete for a manifold method. Unlike a finite collection of unrelated subspaces, the samples $\mathbf{q}_{\text{M}}^{(j)}$ lie on a low-dimensional latent domain. The weights are ultimately intended to define a function

$$\mathbf{q}_{\text{M}} \mapsto \mathbf{w}(\mathbf{q}_{\text{M}}),$$

to be evaluated online by interpolation or regression. Therefore, the vectors $\zeta^{(j)}$ should not vary arbitrarily from one sampled state to its neighbors.

This leads to the smoothness-augmented formal problem

$$\begin{aligned} \min_{\zeta^{(1)}, \dots, \zeta^{(N_M)}} \quad & \left\| \sum_{j=1}^{N_M} \zeta^{(j)} \right\|_0 + \frac{\alpha_G}{2} \sum_{i=1}^{n_{ip}} \zeta_i^T \mathbf{K}_G \zeta_i \\ \text{subject to} \quad & \mathbf{U}_j^T \zeta^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M, \\ & \zeta^{(j)} \geq 0, \quad j = 1, \dots, N_M. \end{aligned}$$

Here $\zeta_i \in \mathbb{R}^{N_M}$ collects the values, over the sampled manifold, of the weight associated with integration point i , and $\mathbf{K}_G$ is a graph Laplacian on the sampled latent states. The graph term penalizes lack of regularity of the weight fields over the manifold.

Problem (3.2) is the full conceptual formulation of MAW-ECM. It contains, in a single statement, the desired common support, local exactness, positivity, and manifold regularity.

3.3 Why the full formulation is not used directly

Although (3.2) is conceptually useful, it is not a practical algorithm. Its difficulty is not merely that it contains many unknowns. The fundamental obstacle is the combination of a combinatorial support selection term with equality constraints, positivity constraints, and a global manifold-smoothness penalty.

The zero-norm term encodes the discrete decision of which integration points are retained. The equality constraints encode the exactness of the reduced integrals at every sampled manifold state. The positivity constraints impose admissibility of the cubature weights. The graph term couples the sampled states and destroys the complete separability that one would otherwise exploit node by node.

Thus, a direct solution of (3.2) would require solving, in one monolithic step, the support-selection problem and the manifold-coupled constrained redistribution problem. This is not the route taken here.

The purpose of (3.2) is instead to clarify the target structure of the method. It identifies what the algorithm should approximate: a sparse common support, with nonnegative weights that vary over the manifold while preserving the local cubature constraints.

3.4 Restriction to an initial ECM support

The first practical simplification is to avoid working on the full set of FE integration points. MAW-ECM starts from an already feasible fixed ECM rule, with support

$$\mathcal{Z}_0 \subset \mathcal{Z}_{\text{FE}}, \quad |\mathcal{Z}_0| = n_c,$$

and weights

$$\mathbf{w}_0 \in \mathbb{R}^{n_c}.$$

The adaptive weights are then defined only on this candidate support:

$$\mathbf{w}^{(j)} \in \mathbb{R}^{n_c}, \quad j = 1, \dots, N_M.$$

This restriction is not merely a computational convenience. It reflects the constructive philosophy of the method: the initial ECM rule provides a feasible global integration rule, and MAW-ECM attempts to prune it further by exploiting state-dependent weights.

The initial adaptive rule is obtained by replication:

$$\mathbf{w}^{(j)} = \mathbf{w}_0, \quad j = 1, \dots, N_M.$$

Thus, before pruning begins, the adaptive rule coincides with the fixed ECM rule at every sampled manifold state. The weight field is constant over the manifold.

The question becomes whether rows of the matrix

$$\mathbf{W} = [\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(N_M)}] \in \mathbb{R}^{n_c \times N_M}$$

can be progressively eliminated while preserving feasibility of the local systems.

3.5 CECM influence: pruning instead of direct sparsification

The second practical simplification is inherited from the Continuous Empirical Cubature Method (CECM). The relevant lesson from CECM is not that the same problem is being solved, but that a difficult sparse cubature construction can be approached by a controlled sequence of pruning operations starting from a feasible rule.

In the present context, one does not attempt to solve the zero-norm problem (3.2) directly. Instead, one proceeds iteratively. At a given pruning stage, a candidate integration point is tentatively removed from the current support. The corresponding row of $\mathbf{W}$ is forced to vanish for all sampled states. The remaining weights are then redistributed so that the local exactness constraints remain satisfied.

If such a redistribution is feasible, the elimination is accepted. If it is not feasible, the candidate is rejected and another candidate is tested. The pruning sequence stops when no further feasible removal is found, or when a prescribed stopping threshold is reached.

This changes the nature of the problem. The original support-selection problem is replaced by a sequence of feasibility-preserving local decisions:

$$\text{remove one point} \quad \longrightarrow \quad \text{restore feasible weights} \quad \longrightarrow \quad \text{accept or reject.}$$

Sparsity is therefore increased monotonically, but only through admissible steps.

3.6 From the full global step to the implemented two-regime algorithm

The formal smoothness-augmented single-step redistribution problem would seek, after removal of a candidate point, a new weight matrix $\mathbf{W}^{\text{new}}$ minimizing a least-change term plus a graph-smoothness term, subject to local exactness and positivity:

$$\begin{aligned} \min_{\mathbf{W}^{\text{new}}} \quad & \frac{1}{2} \|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2 + \frac{\alpha_G}{2} \text{tr} \left[(\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}) \mathbf{K}_G (\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}})^T \right] \\ \text{subject to} \quad & \mathbf{U}_j^T \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M, \\ & \mathbf{w}^{\text{new}(j)} \geq 0, \quad j = 1, \dots, N_M, \\ & W_{pj}^{\text{new}} = 0, \quad j = 1, \dots, N_M. \end{aligned}$$

This is the clean mathematical form of one graph-regularized pruning step.

Nevertheless, solving (3.6) at every pruning iteration is unnecessary during the early part of the reduction. Numerical experience shows that many points can be removed by a simpler equality-only least-change redistribution. In that regime, positivity is not yet active, and the local systems vary smoothly enough over the manifold that smoothness is often obtained without explicitly imposing the graph penalty.

This observation shaped the implemented algorithm. The current MAW-ECM construction uses two regimes:

- (i) a no-enforcement stage, in which candidate eliminations are attempted through inexpensive least-change redistributions without explicit positivity enforcement;
- (ii) a graph-regularized active-set stage, activated once positivity enforcement becomes necessary.

The first regime is efficient and tends to preserve regularity because it avoids active-set clamping. The second regime is more expensive but necessary near the feasibility boundary, where nonnegative weights can no longer be recovered by unconstrained redistribution alone.

In the current implementation, once explicit enforcement is required, the graph-based version is the intended route. The purely local active-set variant is useful historically and diagnostically, but the working MAW–ECM formulation couples the manifold samples through the graph Laplacian whenever positivity must be enforced.

3.7 Role of positivity and active sets

The appearance of negative weights is not a regression failure. It is a structural event in the constrained pruning process. As the support shrinks, the remaining weights must satisfy the same local exactness constraints with fewer degrees of freedom. The equality-only least-change solution may then leave the positive orthant.

When this happens, nonnegativity is enforced by an active-set strategy. A negative weight is clamped to zero and the redistribution problem is solved again on the remaining free variables. In MAW–ECM this clamping is local in the manifold direction:

$$W_{ij} = 0 \quad \text{may be imposed at one manifold node } j,$$

without imposing the same condition at all other nodes. This is distinct from pruning, which is global:

$$W_{pj} = 0 \quad \text{for all } j = 1, \dots, N_M.$$

Keeping these two mechanisms separate is essential. Pruning removes an integration point from the common support. Active-set clamping only restores nonnegativity at specific entries of the weight field.

This distinction also explains why the resulting map

$$\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$$

may be less regular than the underlying mechanical manifold. The weight map is the solution map of a constrained optimization problem. It can change nonsmoothly when active sets change. Graph regularization does not remove this fact; it only selects, among feasible redistributions, those that introduce less roughness across neighboring latent states.

3.8 Current formulation

The final form of the implemented MAW–ECM method can therefore be summarized as follows.

First, a fixed ECM rule provides an admissible initial support and weights. Second, this rule is lifted to the manifold by assigning the same initial weights to every sampled latent state. Third, local exactness systems are built at each sampled state, including the constraints required by the HROM residual and by the selected outputs. Fourth, points are removed one by one. Each removal is accepted only if the remaining weights can be redistributed so as to satisfy the local constraints and, when required, positivity. Fifth, once the final support has been obtained, the sampled adaptive weights are regressed as functions of the latent coordinates.

Thus, the method realizes the original support-union idea of SAW–ECM and the constructive pruning philosophy associated with CECM, but adapts both to the specific needs of a nonlinear manifold HROM. The result is a fixed-support, manifold-adaptive cubature rule:

$$\int_{\Omega_0} f(x, \mathbf{q}_M) d\Omega \approx \sum_{i \in \mathcal{Z}_{\text{red}}} w_i(\mathbf{q}_M) f(x_i, \mathbf{q}_M).$$

The support $\mathcal{Z}_{\text{red}}$ is selected offline and remains fixed online. The weights are functions of the latent coordinates. The graph regularization is not an independent modeling assumption, but a

practical mechanism introduced when positivity enforcement would otherwise generate irregular weight fields.

This historical route is important because it clarifies what MAW-ECM is and what it is not. It is not a black-box regression of quadrature weights. It is not a fully local cubature rule with state-dependent supports. It is not the direct solution of the ideal zero-norm formulation. Rather, it is a feasible pruning approximation to that formulation, built from a fixed ECM rule, enriched with manifold-dependent weights, and regularized only as much as needed to make the adaptive rule usable in the online HROM.

4 Training manifold and local exactness systems

Let the sampled latent manifold be

$$\mathcal{M}_{\text{tr}} = \{\mathbf{q}_{\text{M}}^{(j)}\}_{j=1}^{N_{\text{M}}}, \quad \mathbf{q}_{\text{M}}^{(j)} \in \mathbb{R}^{r_{\text{M}}}.$$

In the current implementation,

$$r_{\text{M}} = 2, \quad \mathbf{q}_{\text{M}}^{(j)} = \begin{bmatrix} q_{\text{M},1}^{(j)} \\ q_{\text{M},2}^{(j)} \end{bmatrix}.$$

The matrix used in the code is

$$\mathbf{Q}_{\text{M}} = \begin{bmatrix} \begin{array}{c} | \\ \mathbf{q}_{\text{M}}^{(1)} \\ | \end{array} & \begin{array}{c} | \\ \mathbf{q}_{\text{M}}^{(2)} \\ | \end{array} & \cdots & \begin{array}{c} | \\ \mathbf{q}_{\text{M}}^{(N_{\text{M}})} \\ | \end{array} \end{bmatrix} \in \mathbb{R}^{r_{\text{M}} \times N_{\text{M}}}.$$

For each sampled latent state $\mathbf{q}_{\text{M}}^{(j)}$, MAW-ECM constructs a local exactness operator

$$\mathbf{U}_j \in \mathbb{R}^{n_{\text{c}} \times r_j},$$

restricted to the current candidate support, and a right-hand side

$$\mathbf{b}_j \in \mathbb{R}^{r_j}.$$

The local adaptive weights $\mathbf{w}^{(j)} \in \mathbb{R}^{n_{\text{c}}}$ must satisfy

$$\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j.$$

When positivity is enforced, the additional condition is

$$\mathbf{w}^{(j)} \geq 0.$$

Equations (4)–(4) define a family of local cubature problems, one for each sampled manifold state. These local problems are coupled only through the requirement that the support be common to all states and, in the graph-regularized version, through the smoothness penalty imposed on the weight fields.

5 Fixed support and adaptive weights

The defining structural choice of MAW-ECM is:

fixed support, adaptive weights.

Thus,

$$\mathcal{Z}_{\text{red}} \quad \text{is independent of } \mathbf{q}_{\text{M}},$$

whereas

$$\mathbf{w}_{\text{red}} = \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}).$$

Equivalently, at the sampled manifold nodes,

$$\mathbf{W}_{\text{red}} = \begin{bmatrix} \left. \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}^{(1)}) \right| & \left. \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}^{(2)}) \right| & \cdots & \left. \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}^{(N_{\text{M}})}) \right| \end{bmatrix}.$$

This distinction is important. MAW-ECM is not an adaptive-support method. The same integration points are evaluated online for all states. Therefore, the finite element bookkeeping remains simple: the online code only needs the final selected points and elements. What changes from state to state is the value of the cubature weights.

A fully local cubature method would allow both

$$\mathcal{Z}_{\text{red}} = \mathcal{Z}_{\text{red}}(\mathbf{q}_{\text{M}}), \quad \mathbf{w}_{\text{red}} = \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}).$$

Such a strategy may produce even smaller local rules, but it would require state-dependent point selection and would complicate online element management. MAW-ECM deliberately avoids this route.

6 Bivariate graph structure

In the current implementation, the latent samples are assumed to lie on a structured two-dimensional grid. The dimensions of the grid are denoted by

$$n_y, n_x, \quad N_{\text{M}} = n_y n_x.$$

The two latent coordinate arrays are obtained by reshaping the rows of $\mathbf{Q}_{\text{M}}$:

$$\begin{aligned} Q_1 &= \text{reshape} \left(\{q_{\text{M},1}^{(j)}\}_{j=1}^{N_{\text{M}}}, n_y, n_x \right), \\ Q_2 &= \text{reshape} \left(\{q_{\text{M},2}^{(j)}\}_{j=1}^{N_{\text{M}}}, n_y, n_x \right). \end{aligned}$$

A quadrilateral mesh is then built in the latent plane. The nodes of this mesh are not physical finite element nodes; they are sampled reduced states. The resulting graph is denoted by

$$\mathcal{G}_{\text{M}} = (\mathcal{M}_{\text{tr}}, \mathcal{E}_{\text{M}}),$$

where $\mathcal{E}_{\text{M}}$ contains the edges induced by the structured quadrilateral connectivity.

The graph Laplacian, or graph stiffness matrix, is denoted by

$$\mathbf{K}_{\text{G}} \in \mathbb{R}^{N_{\text{M}} \times N_{\text{M}}}.$$

For a scalar field $\varphi : \mathcal{M}_{\text{tr}} \rightarrow \mathbb{R}$, collected in a vector $\boldsymbol{\varphi} \in \mathbb{R}^{N_{\text{M}}}$, the quantity

$$\boldsymbol{\varphi}^T \mathbf{K}_{\text{G}} \boldsymbol{\varphi}$$

measures the roughness of the field over the sampled latent graph.

Since each retained integration point defines one scalar weight field over the manifold, graph regularity of the adaptive weights can be measured by

$$\sum_{i=1}^{n_c} \mathbf{w}_i^T \mathbf{K}_{\text{G}} \mathbf{w}_i,$$

where $\mathbf{w}_i \in \mathbb{R}^{N_{\text{M}}}$ collects the values of the i -th integration-point weight over all sampled states.

Specificity of the two-variable implementation. The use of a structured quadrilateral graph is specific to the bivariate implementation. It relies on the possibility of reshaping the sampled manifold states into a Cartesian-like array. For $r_M > 2$, or for unstructured sampling in any dimension, the same formulation remains valid if $\mathbf{K}_G$ is replaced by a graph Laplacian constructed from a neighborhood relation in latent space, for example using nearest neighbors, Delaunay connectivity, radial kernels, or a manifold mesh.

7 Global MAW–ECM algorithm

The following pseudo-algorithm summarizes the logic implemented in `MAW_ECM.genV1.m`. The individual routines invoked in the algorithm are not expanded here; they are identified as separate sections to be developed later.

Algorithm 1 Offline construction of MAW–ECM with fixed support and adaptive weights

Require: Local integrand data $\{\mathbf{A}_j\}_{j=1}^{N_M}$; FE weights $\mathbf{w}^{\text{FE}}$; initial ECM support $\mathcal{Z}_0$; initial ECM weights $\mathbf{w}_0$; latent samples $\mathbf{Q}_M$; offline parameters.

Ensure: Final support $\mathcal{Z}_{\text{red}}$; adaptive weight samples $\mathbf{W}_{\text{red}}$; regression model $\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$.

1: Build the constant scaffold

$$\mathbf{U}_{\text{fix}} \leftarrow \text{orth}(\mathbf{1}).$$

2: Construct the latent graph and graph Laplacian

$$\mathbf{K}_G \leftarrow \text{GraphLaplacian}(\mathbf{Q}_M).$$

3: Build the weighted projector

$$\mathbf{Q} \leftarrow \text{WeightedOrthonormalBasis}(\mathbf{Q}_{\text{all}}, \mathbf{w}^{\text{FE}}).$$

4: Construct the local exactness systems

$$\{\mathbf{U}_j, \mathbf{b}_j\}_{j=1}^{N_M} \leftarrow \text{LocalBasisBuilder}(\mathbf{U}_{\text{fix}}, \mathcal{Z}_0, \{\mathbf{A}_j\}, \mathbf{w}^{\text{FE}}, \mathbf{Q}).$$

5: Initialize the adaptive weights

$$\mathbf{w}^{(j)} \leftarrow \mathbf{w}_0, \quad j = 1, \dots, N_M.$$

Equivalently,

$$\mathbf{W} \leftarrow [\mathbf{w}_0, \dots, \mathbf{w}_0].$$

6: Initialize active and eliminated sets

$$\mathcal{I}_{\text{act}} \leftarrow \{1, \dots, |\mathcal{Z}_0|\}, \quad \mathcal{I}_{\text{elim}} \leftarrow \emptyset.$$

7: **while** stopping criterion is not satisfied **do**

8: Compute aggregate candidate weights

$$R_i = \sum_{j=1}^{N_M} W_{ij}.$$

9: Define positive candidate set

$$\mathcal{I}_+ = \{i \in \mathcal{I}_{\text{act}} : R_i > 0\}.$$

10: Sort candidates in increasing order of R_i .

11: Attempt a no-enforcement pruning step:

$$(\mathbf{W}, \mathcal{I}_{\text{act}}, \mathcal{I}_{\text{elim}}, \text{flag}) \leftarrow \text{NoEnforcementPruning}(\{\mathbf{U}_j\}, \{\mathbf{b}_j\}, \mathbf{W}, \mathcal{I}_{\text{act}}, \mathcal{I}_{\text{elim}}).$$

12: **if** flag = success **then**

13: Commit the accepted elimination.

14: **else**

15: **if** explicit positivity enforcement is enabled **then**

16: Attempt graph-regularized active-set pruning:

$$(\mathbf{W}, \mathcal{I}_{\text{act}}, \mathcal{I}_{\text{elim}}, \text{flag}) \leftarrow \text{GraphBasedPruning}(\{\mathbf{U}_j\}, \{\mathbf{b}_j\}, \mathbf{W}, \mathcal{I}_{\text{act}}, \mathcal{I}_{\text{elim}}, \mathbf{K}_G).$$

17: **else**

18: Set flag = failure.

19: **end if**

20: **if** flag = failure **then**

21: Stop pruning.

22: **end if**

23: **end if**

24: **end while**

25: Extract the final support

$$\mathcal{Z}_{\text{red}} \leftarrow \mathcal{Z}_0(\mathcal{I}_{\text{act}}).$$

26: Extract the final adaptive weights

$$\mathbf{W}_{\text{red}} \leftarrow \mathbf{W}(\mathcal{I}_{\text{act}}, :).$$

27: Regress the weight field

$$\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$$

from the data

$$\left\{ \mathbf{q}_M^{(j)}, \mathbf{W}_{\text{red}}(:, j) \right\}_{j=1}^{N_M}.$$

$$\mathbf{w}_{\text{red}}(\cdot) \leftarrow \text{WeightRegression}(\mathbf{Q}_M, \mathbf{W}_{\text{red}}).$$

28: Validate the adaptive cubature accuracy over the sampled manifold.

Algorithm 1 summarizes the complete offline construction of MAW–ECM. The construction of the local exactness systems is detailed in Section 10; the graph Laplacian construction is described in Section 11; the three pruning stages are presented in Sections 13, 14, and 15; finally, the regression of the adaptive weight fields is discussed in Section 16.

8 Interpretation of the algorithm

The algorithm can be interpreted as a constrained compression of a family of cubature rules. At the beginning, all sampled manifold states share the same fixed ECM weights:

$$\mathbf{w}^{(1)} = \mathbf{w}^{(2)} = \dots = \mathbf{w}^{(N_M)} = \mathbf{w}_0.$$

As pruning proceeds, the support is reduced, but the equality constraints are maintained by allowing the columns of $\mathbf{W}$ to separate from one another. Thus, the algorithm trades a reduction in the number of integration points for an increase in the complexity of the weight map

$$\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M).$$

This trade-off is the essential mechanism of MAW–ECM. A fixed ECM rule must find one vector of weights that works globally over the whole manifold. MAW–ECM instead constructs a family of related cubature rules:

$$\left\{ \mathcal{Z}_{\text{red}}, \mathbf{w}_{\text{red}}(\mathbf{q}_M^{(j)}) \right\}_{j=1}^{N_M},$$

with common support but state-dependent weights.

The graph-regularized stage is introduced because the independent local redistribution of weights may generate highly irregular weight fields. The graph term couples neighboring latent states during the redistribution step and discourages sharp jumps. However, it should not be interpreted as a guarantee of global smoothness. The map $\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$ is the solution map of a constrained optimization problem with active-set changes, and such maps are generally only piecewise smooth.

9 Specific and general components

It is useful to separate the algorithm into components that are intrinsic to MAW–ECM and components that are specific to the present bivariate implementation.

9.1 General MAW–ECM components

The following components are independent of the latent dimension:

- the fixed-support/adaptive-weight formulation;
- the local exactness constraints $\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j$;
- the global pruning of integration points;
- the no-enforcement least-change redistribution;
- the positivity-constrained active-set redistribution;
- the final regression of $\mathbf{w}_{\text{red}}(\mathbf{q}_M)$;
- the validation of the reduced adaptive cubature rule.

9.2 Components specific to the two-variable implementation

The following components are specific to the present bivariate implementation:

- the assumption that $\mathbf{Q}_M$ can be reshaped into an $n_y \times n_x$ structured grid;
- the construction of a quadrilateral mesh in the latent plane;
- the finite-element-like graph Laplacian built on this quadrilateral mesh;
- the use of two-dimensional plotting and surface visualization of the adaptive weights;
- any hard-coded interpretation of the two latent variables as axial and shear-like macroscopic parameters.

9.3 Extension to higher-dimensional latent manifolds

For $r_M > 2$, the algebraic structure of the method does not change. The local constraints remain

$$\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M,$$

and the adaptive weights are still stored as

$$\mathbf{W} \in \mathbb{R}^{n_c \times N_M}.$$

The only essential modification concerns the construction of $\mathbf{K}_G$. Instead of a structured quadrilateral mesh in the $(q_{M,1}, q_{M,2})$ plane, one needs a graph on the sampled latent states in $\mathbb{R}^{r_M}$. Once such a graph is available, the same graph energy

$$\sum_{i=1}^{n_c} \mathbf{w}_i^T \mathbf{K}_G \mathbf{w}_i$$

can be used without changing the pruning formulation.

Thus, the two-dimensional implementation should be viewed as a convenient realization of a more general graph-based MAW-ECM principle.

10 Construction of the local exactness systems

The first substantive operation in MAW-ECM is the construction of the local linear systems that the adaptive weights must satisfy. These systems are the manifold-dependent counterpart of the ECM exactness condition. In the classical fixed-weight setting, a single cubature vector is sought so as to integrate a global reduced basis of integrands. In MAW-ECM, by contrast, each sampled manifold state carries its own local exactness system, while all systems share the same candidate support.

Let the initial ECM rule be given by the support

$$\mathcal{Z}_0 = \{z_1, \dots, z_{n_c}\} \subset \mathcal{Z}_{FE},$$

and by the corresponding fixed weights

$$\mathbf{w}_0 \in \mathbb{R}^{n_c}.$$

At this stage, $\mathcal{Z}_0$ is not yet the final MAW-ECM support. It is the candidate set from which the adaptive pruning algorithm will remove points.

For each sampled manifold state $\mathbf{q}_M^{(j)}$, let

$$\mathbf{A}_j \in \mathbb{R}^{n_{ip} \times m_j}$$

denote the local integrand matrix associated with that state. The columns of $\mathbf{A}_j$ contain the scalar fields over the full integration grid that one wishes to integrate accurately at $\mathbf{q}_M^{(j)}$. Depending on the application, these columns may represent stress-output integrands, residual integrands projected onto a manifold tangent space, or a concatenation of several blocks.

The full finite element integral of these fields is

$$\mathbf{A}_j^T \mathbf{w}^{\text{FE}}.$$

However, the MAW–ECM pruning stage acts only on the initial candidate support $\mathcal{Z}_0$. Therefore, the local basis is restricted to the rows indexed by $\mathcal{Z}_0$:

$$\mathbf{U}_j = \mathbf{U}_{\text{full}j}(\mathcal{Z}_0, :) \in \mathbb{R}^{n_c \times r_j},$$

where $\mathbf{U}_{\text{full}j}$ denotes the orthonormalized local basis before restriction, and r_j is the number of local constraints retained at node j .

The local adaptive weights $\mathbf{w}^{(j)}$ are required to satisfy

$$\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M.$$

The pair $(\mathbf{U}_j, \mathbf{b}_j)$ is the fundamental local algebraic object used by all subsequent pruning routines.

10.1 Fixed scaffold

A fixed scaffold is included in every local basis. Its role is to impose constraints that must be preserved independently of the state-dependent integrand content. In the present implementation, the most important scaffold is the constant function,

$$\mathbf{1} = (1, \dots, 1)^T \in \mathbb{R}^{n_{\text{ip}}}.$$

After orthonormalization, this gives

$$\mathbf{U}_{\text{fix}} = \text{orth}(\mathbf{1}).$$

The corresponding constraint enforces preservation of the total measure carried by the current cubature rule. In the candidate-support formulation, this means that the adaptive weights should preserve the volume represented by the initial ECM rule:

$$\mathbf{1}_{\mathcal{Z}_0}^T \mathbf{w}^{(j)} = \mathbf{1}_{\mathcal{Z}_0}^T \mathbf{w}_0, \quad j = 1, \dots, N_M.$$

Thus, even if the state-dependent integrand block becomes small or degenerate, the adaptive rule is prevented from collapsing to the trivial zero solution.

More general scaffolds are possible. For instance, one may include additional low-order functions, elastic modes, or previously identified integration constraints. In that case,

$$\mathbf{U}_{\text{fix}} = \text{orth}([\mathbf{U}_{\text{el}} \ \mathbf{1}]),$$

where $\mathbf{U}_{\text{el}}$ denotes a user-defined fixed block. The present bivariate implementation uses only the constant scaffold.

10.2 Weighted projector

The routine also constructs a weighted projector from a precomputed basis $\mathbf{Q}_{\text{all}}$. This object is used to project or filter the local integrand matrices before building the local exactness bases.

Let

$$\mathbf{Q}_{\text{all}} \in \mathbb{R}^{n_{\text{ip}} \times n_Q}$$

denote the input basis. The code forms

$$\mathbf{Q} = \text{orth} \left(\text{diag}(\mathbf{w}^{\text{FE}})^{-1/2} \mathbf{Q}_{\text{all}} \right).$$

Equivalently, if

$$\mathbf{S}_{\text{FE}} = \text{diag}(\sqrt{w_1^{\text{FE}}}, \dots, \sqrt{w_{n_{\text{ip}}}^{\text{FE}}}),$$

then

$$\mathbf{Q} = \text{orth} \left(\mathbf{S}_{\text{FE}}^{-1} \mathbf{Q}_{\text{all}} \right).$$

This transformation is intended to work with a basis compatible with the finite-element weighted inner product. In implementation terms, this is the operation

$$\mathbf{Q} \leftarrow \text{SVD}(\mathbf{Q}_{\text{all}} / \sqrt{\mathbf{w}^{\text{FE}}}).$$

When the projection option is active, the local integrand matrix is replaced by

$$\mathbf{A}_j^{\text{proj}} = \mathbf{Q}(\mathbf{Q}^T \mathbf{A}_j).$$

This projected content is then used to construct the local basis. The purpose is not to change the cubature philosophy, but to avoid imposing local constraints associated with high-frequency or poorly represented components of the integrand snapshots.

10.3 Local basis at a sampled manifold state

For each sampled state j , one starts from the local integrand block $\mathbf{A}_j$. If the projection option is active, the projected matrix $\mathbf{A}_j^{\text{proj}}$ is used instead. For compactness, denote the matrix actually used by

$$\hat{\mathbf{A}}_j.$$

Thus,

$$\hat{\mathbf{A}}_j = \begin{cases} \mathbf{Q}(\mathbf{Q}^T \mathbf{A}_j), & \text{if projection is active,} \\ \mathbf{A}_j, & \text{otherwise.} \end{cases}$$

The local full-grid basis is then obtained by orthonormalizing the concatenation

$$\mathbf{U}_{\text{full}j} = \text{orth} \left([\mathbf{U}_{\text{fix}} \quad \hat{\mathbf{A}}_j] \right).$$

In the implementation this orthonormalization is performed by **SVD**. Only the rows associated with the initial ECM support are retained:

$$\mathbf{U}_j = \mathbf{U}_{\text{full}j}(\mathcal{Z}_0, :).$$

The local right-hand side depends on whether projection is active. Without projection, the natural target is the full FE integral:

$$\mathbf{b}_j = \mathbf{U}_{\text{full}j}^T \mathbf{w}^{\text{FE}}.$$

With projection, the implementation instead uses the integral induced by the current fixed ECM rule:

$$\mathbf{b}_j = \mathbf{U}_j^T \mathbf{w}_0.$$

This distinction is important. In the projected case, the local system is made consistent with the current candidate rule. The pruning problem then asks whether the same projected constraints can still be satisfied after removing additional points from $\mathcal{Z}_0$.

10.4 Initialization of the adaptive weights

Once the local systems have been constructed, the adaptive weight matrix is initialized by copying the fixed ECM weights at every sampled manifold state:

$$\mathbf{w}^{(j)} = \mathbf{w}_0, \quad j = 1, \dots, N_M.$$

Equivalently,

$$\mathbf{W} = [\mathbf{w}_0, \mathbf{w}_0, \dots, \mathbf{w}_0] \in \mathbb{R}^{n_c \times N_M}.$$

Thus, the initial MAW–ECM rule is exactly the fixed ECM rule replicated over the sampled manifold. Adaptivity is introduced only through the subsequent pruning and redistribution steps.

This initialization has two useful consequences. First, feasibility is inherited from the initial rule, up to the consistency of the constructed local systems. Second, the pruning algorithm starts from a state in which the weight field is constant over the manifold:

$$\mathbf{w}(\mathbf{q}_M^{(1)}) = \dots = \mathbf{w}(\mathbf{q}_M^{(N_M)}).$$

All spatial variability of the adaptive weights is therefore generated by the need to preserve the local exactness constraints after points are removed.

10.5 Function-level interpretation

The construction described in this section is implemented in

`MAWecmBasisMATRIX_LOCAL_noverlap.m`.

Its role is purely preparatory: it does not prune integration points and does not regress the adaptive weights. It builds the local systems

$$\{(\mathbf{U}_j, \mathbf{b}_j)\}_{j=1}^{N_M}$$

and initializes $\mathbf{W}$.

In pseudo-code, the function performs the following operations.

Algorithm 2 Construction of local exactness systems

Require: Fixed scaffold $\mathbf{U}_{\text{fix}}$; initial support $\mathcal{Z}_0$; local integrand matrices $\{\mathbf{A}_j\}_{j=1}^{N_M}$; initial weights $\mathbf{w}_0$; FE weights $\mathbf{w}^{\text{FE}}$; optional projector $\mathbf{Q}$.

Ensure: Local operators $\{\mathbf{U}_j\}$; right-hand sides $\{\mathbf{b}_j\}$; initial adaptive weights $\mathbf{W}$.

1: **for** $j = 1, \dots, N_M$ **do**

2: **if** projection through $\mathbf{Q}$ is active **then**

3: Compute

$$\hat{\mathbf{A}}_j = \mathbf{Q}(\mathbf{Q}^T \mathbf{A}_j).$$

4: **else**

5: Set

$$\hat{\mathbf{A}}_j = \mathbf{A}_j.$$

6: **end if**

7: Build the full local basis

$$\mathbf{U}_{\text{full}j} = \text{orth} \left([\mathbf{U}_{\text{fix}} \quad \hat{\mathbf{A}}_j] \right).$$

8: Restrict the basis to the current candidate support

$$\mathbf{U}_j = \mathbf{U}_{\text{full}j}(\mathcal{Z}_0, :).$$

9: **if** projection through $\mathbf{Q}$ is active **then**

10: Define the right-hand side by the current ECM rule

$$\mathbf{b}_j = \mathbf{U}_j^T \mathbf{w}_0.$$

11: **else**

12: Define the right-hand side by the full FE quadrature

$$\mathbf{b}_j = \mathbf{U}_{\text{full}j}^T \mathbf{w}^{\text{FE}}.$$

13: **end if**

14: Initialize

$$\mathbf{w}^{(j)} = \mathbf{w}_0.$$

15: **end for**

16: Assemble

$$\mathbf{W} = [\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(N_M)}].$$

10.6 Specificity and possible generalizations

The construction of the local exactness systems is not specific to a two-dimensional latent manifold. The only required input is a collection of sampled states indexed by $j = 1, \dots, N_M$. Therefore, the equations

$$\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j$$

remain unchanged for any latent dimension r_M .

What is specific to the present implementation is the organization of the samples and the later use of a structured bivariate graph. The local systems themselves do not know whether the manifold is one-dimensional, two-dimensional, or higher-dimensional. They only require a consistent ordering between:

$$\mathbf{q}_M^{(j)}, \quad \mathbf{A}_j, \quad \mathbf{U}_j, \quad \mathbf{b}_j, \quad \mathbf{w}^{(j)}.$$

This observation is important for future extensions. In higher-dimensional or unstructured

latent spaces, the present section would remain essentially unchanged. Only the graph construction and visualization tools would need to be modified.

11 Graph Laplacian on the sampled latent manifold

The graph-based version of MAW–ECM requires a discrete notion of smoothness for scalar fields defined over the sampled latent manifold. In the present setting, these scalar fields are the adaptive cubature weights associated with the retained integration points. For a fixed integration point i , the values

$$W_{i1}, W_{i2}, \dots, W_{iN_M}$$

define a scalar field over the sampled latent states

$$\{\mathbf{q}_M^{(j)}\}_{j=1}^{N_M}.$$

The purpose of the graph Laplacian is to measure and penalize nonsmooth variations of such fields across neighboring latent states.

The graph construction is independent of the physical finite element mesh. It is built in the latent space, not in the spatial domain. The physical mesh defines the integration points; the latent graph defines neighborhood relations between sampled reduced states.

11.1 Structured bivariate graph

In the current implementation, the latent dimension is two:

$$r_M = 2, \quad \mathbf{q}_M = \begin{bmatrix} q_{M,1} \\ q_{M,2} \end{bmatrix}.$$

The sampled states are assumed to be ordered as a structured grid. Let

$$n_y n_x = N_M$$

be the number of samples in the two grid directions. The two rows of $\mathbf{Q}_M$ are reshaped into two arrays

$$Q_1 = \text{reshape} \left(q_{M,1}^{(1)}, \dots, q_{M,1}^{(N_M)}; n_y, n_x \right),$$

and

$$Q_2 = \text{reshape} \left(q_{M,2}^{(1)}, \dots, q_{M,2}^{(N_M)}; n_y, n_x \right).$$

Each entry (a, b) of these arrays defines one node of the latent graph:

$$\mathbf{x}_{ab}^G = \begin{bmatrix} Q_1(a, b) \\ Q_2(a, b) \end{bmatrix}.$$

A structured quadrilateral mesh is then built over these nodes. Its connectivity is denoted by

$$\mathcal{T}_G.$$

Again, this is not a finite element mesh in the physical domain. It is an auxiliary mesh over the sampled latent states.

11.2 Finite-element-like graph stiffness

The graph Laplacian $\mathbf{K}_G$ is assembled as a scalar diffusion stiffness matrix on the auxiliary latent mesh. Formally, if φ is a scalar field defined over the latent manifold, one may view its nodal values as

$$\boldsymbol{\varphi} = \begin{bmatrix} \varphi(\mathbf{q}_M^{(1)}) & \cdots & \varphi(\mathbf{q}_M^{(N_M)}) \end{bmatrix}^T.$$

The discrete graph energy is then

$$\mathcal{E}_G(\boldsymbol{\varphi}) = \boldsymbol{\varphi}^T \mathbf{K}_G \boldsymbol{\varphi}.$$

This quantity is the discrete counterpart of a Dirichlet energy in latent space,

$$\int_{\mathcal{M}} \nabla_{\mathbf{q}_M} \varphi \cdot \nabla_{\mathbf{q}_M} \varphi d\mathbf{q}_M.$$

Thus, $\mathcal{E}_G$ is small for slowly varying fields and large for fields with strong jumps between neighboring latent states.

In MAW-ECM, the scalar fields of interest are the rows of the adaptive weight matrix. Let

$$\mathbf{w}_i = [W_{i1} \quad W_{i2} \quad \cdots \quad W_{iN_M}]^T \in \mathbb{R}^{N_M}$$

be the weight field associated with integration point i . The total graph roughness of the adaptive rule is measured by

$$\mathcal{S}_G(\mathbf{W}) = \sum_{i=1}^{n_c} \mathbf{w}_i^T \mathbf{K}_G \mathbf{w}_i.$$

Equivalently, since the rows of $\mathbf{W}$ are the weight fields,

$$\mathcal{S}_G(\mathbf{W}) = \text{tr}(\mathbf{W} \mathbf{K}_G \mathbf{W}^T).$$

11.3 Incremental smoothing

In the pruning step, it is usually preferable to regularize the change in the weights rather than the absolute value of the weights. Let

$$\Delta \mathbf{W} = \mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}$$

be the redistribution induced by a tentative point removal. The graph regularization term used in the update can then be written as

$$\mathcal{S}_G(\Delta \mathbf{W}) = \text{tr}(\Delta \mathbf{W} \mathbf{K}_G \Delta \mathbf{W}^T).$$

This choice has a clear interpretation. The previous feasible weights $\mathbf{W}^{\text{old}}$ are taken as a reference state. The update is allowed to change them, but abrupt changes over neighboring latent states are penalized.

A representative graph-regularized least-change objective is therefore

$$\frac{1}{2} \|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2 + \frac{\alpha_G}{2} \text{tr}[(\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}) \mathbf{K}_G (\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}})^T],$$

where $\alpha_G \geq 0$ controls the strength of the graph regularization. For $\alpha_G = 0$, one recovers a purely local least-change redistribution. As α_G increases, rough variations of the redistribution over the latent graph become increasingly penalized.

11.4 Stacked-vector form

For the graph-regularized active-set update, it is useful to write (11.3) in vector form. Let

$$\mathbf{z} = \text{vec}(\mathbf{W}^{\text{new}}), \quad \mathbf{z}^{\text{old}} = \text{vec}(\mathbf{W}^{\text{old}}),$$

where the columns of the weight matrix are stacked. Then

$$\|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2 = \|\mathbf{z} - \mathbf{z}^{\text{old}}\|_2^2.$$

Moreover,

$$\text{tr} \left[(\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}) \mathbf{K}_G (\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}})^T \right] = (\mathbf{z} - \mathbf{z}^{\text{old}})^T (\mathbf{K}_G \otimes \mathbf{I}_{n_c}) (\mathbf{z} - \mathbf{z}^{\text{old}}).$$

Consequently, the incremental graph-regularized objective becomes

$$\frac{1}{2} (\mathbf{z} - \mathbf{z}^{\text{old}})^T [\mathbf{I} + \alpha_G (\mathbf{K}_G \otimes \mathbf{I}_{n_c})] (\mathbf{z} - \mathbf{z}^{\text{old}}).$$

Define

$$\mathbf{H}_G := \mathbf{I} + \alpha_G (\mathbf{K}_G \otimes \mathbf{I}_{n_c}).$$

Then the objective is simply

$$\frac{1}{2} (\mathbf{z} - \mathbf{z}^{\text{old}})^T \mathbf{H}_G (\mathbf{z} - \mathbf{z}^{\text{old}}).$$

The structure of the Kronecker product is important. The Laplacian $\mathbf{K}_G$ acts across manifold samples, whereas $\mathbf{I}_{n_c}$ acts on the integration-point index. Therefore, the graph regularization does not mix different integration points. Each weight field is smoothed independently over the manifold graph.

11.5 Boundary-weighted graph regularization

The implementation also allows the graph Laplacian to be modified near the boundary of the sampled latent domain. This option increases the regularization strength close to the boundary, where interpolation and extrapolation effects are often more delicate.

Let d_j denote the grid-index distance from node j to the closest boundary of the structured latent grid. A nodal amplification factor is defined as

$$\rho_j = 1 + \lambda_B \exp \left(-\frac{d_j}{\ell_B} \right), \quad \rho_j \geq 1.$$

Here λ_B controls the strength of the boundary amplification, and ℓ_B controls its decay length.

For an edge connecting nodes a and b , the edge amplification is taken as

$$\rho_{ab} = \frac{1}{2} (\rho_a + \rho_b).$$

The original graph edge weight is then multiplied by ρ_{ab} , and the Laplacian is rebuilt from the modified edge weights. The resulting matrix is a boundary-weighted graph Laplacian. Its purpose is not to change the exactness constraints, but only to alter the smoothness metric used during graph-based redistribution.

11.6 Specificity and extension to higher dimensions

The construction based on Q_1 , Q_2 and quadrilateral connectivity is specific to the bivariate implementation. It requires that the latent samples can be reshaped into a structured two-dimensional grid.

The graph-regularized MAW–ECM formulation itself does not require this. For a general latent dimension r_M , one only needs a graph

$$\mathcal{G}_M = (\mathcal{M}_{\text{tr}}, \mathcal{E}_M)$$

whose nodes are the sampled latent states. The graph Laplacian can then be constructed from any suitable neighborhood relation. Possible choices include:

k -nearest-neighbor graphs, ε -graphs, radial-kernel graphs, simplicial or manifold meshes.

Once $\mathbf{K}_G$ is available, the same graph energy

$$\text{tr}(\mathbf{W} \mathbf{K}_G \mathbf{W}^T)$$

and the same stacked operator

$$\mathbf{I} + \alpha_G (\mathbf{K}_G \otimes \mathbf{I})$$

apply without modification.

Thus, the bivariate graph used in the present implementation should be viewed as one convenient realization of a more general graph-based smoothing principle for manifold-adaptive cubature weights.

11.7 Function-level interpretation

The construction of the bivariate graph is split into small auxiliary routines. The function

`quadMeshFromXY.m`

constructs the nodal coordinate array and quadrilateral connectivity from the reshaped latent-coordinate arrays Q_1 and Q_2 . The function

`ComputeK_scl.m`

assembles the scalar finite-element-like graph stiffness matrix $\mathbf{K}_G$ on this auxiliary latent mesh. If boundary amplification is activated,

`BoundaryWeightedGraphLaplacian.m`

modifies the edge weights and rebuilds the Laplacian.

The resulting matrix $\mathbf{K}_G$ is then passed to the graph-based pruning routine, where it enters the regularized redistribution of the adaptive weights.

12 Pruning strategy and candidate elimination

Once the local exactness systems and the graph operator have been constructed, MAW–ECM enters the pruning stage. The objective is to reduce the initial candidate support $\mathcal{Z}_0$ while preserving, at every sampled manifold state, the local cubature constraints

$$\mathbf{U}_j^T \mathbf{w}^{(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M.$$

The reduction is performed by eliminating integration points one at a time. Each elimination removes one row of the adaptive weight matrix

$$\mathbf{W} \in \mathbb{R}^{n_c \times N_M}.$$

Thus, pruning is global with respect to the support: if a point is eliminated, it is eliminated for all manifold states.

The adaptive character of the method enters through the redistribution of the remaining weights. After a point is removed, each column of $\mathbf{W}$ is allowed to change so that the corresponding local exactness system remains satisfied.

12.1 Active and eliminated sets

Let

$$\mathcal{I}_{\text{act}} \subset \{1, \dots, n_c\}$$

denote the set of currently active candidate indices, and let

$$\mathcal{I}_{\text{elim}} = \{1, \dots, n_c\} \setminus \mathcal{I}_{\text{act}}$$

denote the eliminated indices. At the beginning of the pruning stage,

$$\mathcal{I}_{\text{act}} = \{1, \dots, n_c\}, \quad \mathcal{I}_{\text{elim}} = \emptyset.$$

If $p \in \mathcal{I}_{\text{act}}$ is selected for elimination, the updated active set is

$$\mathcal{I}_{\text{act}}^+ = \mathcal{I}_{\text{act}} \setminus \{p\},$$

and the eliminated set becomes

$$\mathcal{I}_{\text{elim}}^+ = \mathcal{I}_{\text{elim}} \cup \{p\}.$$

In terms of adaptive weights, the elimination means

$$W_{pj}^+ = 0, \quad j = 1, \dots, N_M.$$

The remaining rows of $\mathbf{W}$ must then be recomputed.

12.2 Candidate ranking by aggregate weight

The first candidate ranking used in the algorithm is based on the aggregate contribution of each integration point over the sampled manifold. For a current weight matrix $\mathbf{W}$, define

$$R_i = \sum_{j=1}^{N_M} W_{ij}, \quad i = 1, \dots, n_c.$$

Only candidates with positive aggregate contribution are considered:

$$\mathcal{I}_+ = \{i \in \mathcal{I}_{\text{act}} : R_i > 0\}.$$

The candidates in $\mathcal{I}_+$ are sorted in increasing order of R_i . Hence, the algorithm first attempts to remove the integration points that carry the smallest total weight over the manifold.

This criterion is intentionally simple. It is the natural extension of the greedy intuition used in fixed-weight cubature: points with small cumulative weight are expected to be less important. In MAW-ECM, however, a small value of R_i does not guarantee that the point can be safely removed. A point may have small total weight and still be essential for satisfying one or several local exactness systems.

12.3 Single-point elimination trial

Consider a candidate point $p \in \mathcal{I}_+$. A pruning trial consists of imposing

$$W_{pj}^+ = 0, \quad j = 1, \dots, N_M,$$

and seeking updated weights $\mathbf{W}^{\text{new}}$ such that

$$\mathbf{U}_j^T \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M.$$

If positivity is enforced, one also requires

$$\mathbf{w}^{\text{new}(j)} \geq 0, \quad j = 1, \dots, N_M.$$

A candidate is accepted only if the redistribution problem is feasible according to the active stage of the algorithm. Otherwise, the point is rejected and the next candidate in the ordered list is tested.

The important point is that at most one integration point is committed per successful pruning pass. This conservative strategy makes the evolution of the support easier to monitor and reduces the risk of destroying feasibility by an overly aggressive simultaneous elimination.

12.4 Stopping criterion

The pruning loop terminates when either the target number of points has been reached or no further feasible elimination can be found. A natural lower bound is dictated by the number of local constraints. If

$$r_j = \dim(\mathbf{b}_j),$$

then a necessary condition for local feasibility is that the number of active points should not fall below the number of independent constraints at the most demanding node. Hence one defines

$$r_{\max} = \max_{1 \leq j \leq N_M} r_j.$$

In practice, an additional user-prescribed maximum or target number of ECM points may also be imposed. Denoting this value by n_{target} , the effective stopping threshold can be written as

$$n_{\text{stop}} = \max(r_{\max}, n_{\text{target}}).$$

The pruning loop is not allowed to reduce the support below this threshold.

Even above this threshold, pruning may stop earlier if no candidate produces a feasible redistribution. This is a stronger and more meaningful stopping criterion than the mere count of active points.

12.5 Two-stage pruning logic

The implementation distinguishes two main redistribution stages.

The first stage attempts a least-change update without explicit positivity enforcement. This stage is computationally inexpensive and usually produces smooth changes in the adaptive weights. Symbolically, for a candidate point p , it resembles

$$\min_{\mathbf{W}^{\text{new}}} \frac{1}{2} \|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2,$$

subject to

$$\mathbf{U}_j^T \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j, \quad W_{pj}^+ = 0, \quad j = 1, \dots, N_M.$$

No inequality constraint is imposed at this stage.

If this no-enforcement stage can no longer eliminate points, the algorithm may switch to an explicit positivity stage. In that case, the redistribution also enforces

$$\boldsymbol{w}^{\text{new}(j)} \geq 0.$$

This positivity-enforcing stage can be performed either independently at each manifold node, or through the graph-regularized coupled formulation described later.

The transition between these stages is important. The no-enforcement stage usually preserves a relatively regular weight field. The explicit positivity stage is more robust from the cubature viewpoint, but it introduces active-set changes, and these active-set changes are a primary source of nonsmoothness in the mapping

$$\boldsymbol{q}_M \mapsto \boldsymbol{w}_{\text{red}}(\boldsymbol{q}_M).$$

12.6 Pseudo-algorithm for the pruning loop

The pruning loop can be summarized as follows.

Algorithm 3 MAW–ECM pruning loop

Require: Local systems $\{(\mathbf{U}_j, \mathbf{b}_j)\}_{j=1}^{N_M}$; initial adaptive weights $\mathbf{W}$; initial support $\mathcal{Z}_0$; graph Laplacian $\mathbf{K}_G$; stopping threshold n_{stop} .

Ensure: Reduced support $\mathcal{Z}_{\text{red}}$; adaptive weights on the reduced support.

1: Initialize

$$\mathcal{I}_{\text{act}} \leftarrow \{1, \dots, |\mathcal{Z}_0|\}, \quad \mathcal{I}_{\text{elim}} \leftarrow \emptyset.$$

2: **while** $|\mathcal{I}_{\text{act}}| > n_{\text{stop}}$ **do**

3: Compute aggregate weights

$$R_i = \sum_{j=1}^{N_M} W_{ij}.$$

4: Define the candidate list

$$\mathcal{I}_+ = \{i \in \mathcal{I}_{\text{act}} : R_i > 0\}.$$

5: Sort $\mathcal{I}_+$ in increasing order of R_i .

6: Attempt no-enforcement pruning over the ordered candidates.

7: **if** a feasible candidate is found **then**

8: Commit the elimination:

$$\mathcal{I}_{\text{act}} \leftarrow \mathcal{I}_{\text{act}} \setminus \{p\}, \quad \mathcal{I}_{\text{elim}} \leftarrow \mathcal{I}_{\text{elim}} \cup \{p\}.$$

9: Update $\mathbf{W}$.

10: **else**

11: **if** positivity enforcement is disabled **then**

12: Stop.

13: **else**

14: **if** graph-based redistribution is enabled **then**

15: Attempt graph-regularized active-set pruning.

16: **else**

17: Attempt local active-set pruning independently at each node.

18: **end if**

19: **if** a feasible candidate is found **then**

20: Commit the elimination and update $\mathbf{W}$.

21: **else**

22: Stop.

23: **end if**

24: **end if**

25: **end if**

26: **end while**

27: Set

$$\mathcal{Z}_{\text{red}} = \mathcal{Z}_0(\mathcal{I}_{\text{act}}).$$

28: Keep

$$\mathbf{W}_{\text{red}} = \mathbf{W}(\mathcal{I}_{\text{act}}, :).$$

12.7 Interpretation

The pruning process should not be viewed as simply deleting points with small weights. Each deletion changes the feasible set of all local cubature systems. A point can be removed only if the remaining support can still reproduce the local constraints at every sampled manifold state.

Thus, MAW–ECM performs a constrained sparsification of a family of cubature rules. The

support is sparse and common, but the weights are allowed to move over the manifold. This additional freedom is precisely what allows the support to be smaller than in a fixed-weight ECM rule.

At the same time, this freedom introduces a new difficulty. The adaptive weight field is the result of repeated constrained redistributions. When positivity is enforced, changes of active set may produce kinks, jumps, or localized spikes. The graph-based stage is introduced to moderate these effects, but the underlying constrained nature of the problem remains.

12.8 Function-level interpretation

The global pruning loop is orchestrated by

`MAW_ECM_genV1.m.`

The no-enforcement stage is implemented in

`Loop_MAWecmNOENF_cl.m.`

The explicit positivity-enforcing local stage is implemented in

`Loop_MAWecmNOENFe.m.`

The graph-based positivity-enforcing stage is implemented in

`GraphBased_MAW_ECM_2v.m,`

which in turn calls

`prune_step_optionB.m.`

The detailed mathematical formulation of these redistribution stages is not included in the present section. It will be developed in the following sections, starting from the no-enforcement least-change update and then moving to the active-set and graph-regularized formulations.

13 Stage 1: no-enforcement least-change redistribution

The first redistribution stage of MAW-ECM attempts to remove integration points without explicitly enforcing positivity of the adaptive weights. This stage is implemented in

`Loop_MAWecmNOENF_cl.m.`

Its role is to perform inexpensive pruning steps while preserving the local exactness constraints with minimal perturbation of the current adaptive weights.

This stage is important for two reasons. First, it is computationally cheaper than the active-set positivity-enforcing stages. Second, and more importantly, it tends to preserve smoothness of the adaptive weight field. Since no weights are explicitly clamped to zero, the redistribution remains governed only by the equality constraints, and the resulting changes in the weights are usually relatively gentle over the latent manifold.

The no-enforcement stage should therefore be interpreted as the preferred redistribution regime. The explicit positivity stages are activated only when the unconstrained redistribution is no longer able to reduce the support.

13.1 Local least-change redistribution

Suppose that the current candidate support contains n_c active points and that a candidate point p is selected for elimination. The pruning condition is

$$W_{pj}^+ = 0, \quad j = 1, \dots, N_M.$$

The remaining adaptive weights must then be redistributed so as to preserve the local exactness systems

$$\mathbf{U}_j^T \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j, \quad j = 1, \dots, N_M.$$

At the no-enforcement stage, the redistribution is performed independently at each manifold sample.

Let

$$\mathbf{w}^{\text{old}(j)}$$

denote the current local adaptive weights at node j . The updated local weights are obtained by solving a least-change problem of the form

$$\min_{\mathbf{w}^{\text{new}(j)}} \frac{1}{2} \|\mathbf{w}^{\text{new}(j)} - \mathbf{w}^{\text{old}(j)}\|_2^2,$$

subject to

$$\mathbf{U}_j^T \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j,$$

and

$$w_{p,\text{new}}^{(j)} = 0.$$

No positivity condition is imposed:

$$\mathbf{w}^{\text{new}(j)} \not\geq 0 \quad \text{necessarily.}$$

Thus, the update seeks the feasible redistribution closest, in Euclidean norm, to the previous local weights.

13.2 Reduced local system after elimination

Let

$$\mathcal{I}_{\text{rem}} = \mathcal{I}_{\text{act}} \setminus \{p\}$$

denote the remaining active rows after tentative elimination of point p . Define the reduced local basis

$$\widetilde{\mathbf{U}}_j = \mathbf{U}_j(\mathcal{I}_{\text{rem}}, :) \in \mathbb{R}^{(n_c-1) \times r_j}.$$

Similarly, define the reduced local unknown

$$\widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{w}^{\text{new}(j)}(\mathcal{I}_{\text{rem}}).$$

Since the eliminated weight is fixed to zero, the local exactness condition reduces to

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j.$$

The least-change problem therefore becomes

$$\min_{\widetilde{\mathbf{w}}^{\text{new}(j)}} \frac{1}{2} \|\widetilde{\mathbf{w}}^{\text{new}(j)} - \widetilde{\mathbf{w}}^{\text{old}(j)}\|_2^2,$$

subject to

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j.$$

13.3 Closed-form least-change update

The problem (13.2) is a standard equality-constrained quadratic minimization problem. Introducing Lagrange multipliers

$$\lambda_j \in \mathbb{R}^{r_j},$$

the local Lagrangian reads

$$\mathcal{L} = \frac{1}{2} \|\widetilde{\mathbf{w}}^{\text{new}(j)} - \widetilde{\mathbf{w}}^{\text{old}(j)}\|_2^2 + \lambda_j^T \left(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} - \mathbf{b}_j \right).$$

Stationarity with respect to $\widetilde{\mathbf{w}}^{\text{new}(j)}$ gives

$$\widetilde{\mathbf{w}}^{\text{new}(j)} - \widetilde{\mathbf{w}}^{\text{old}(j)} + \widetilde{\mathbf{U}}_j \lambda_j = 0.$$

Hence,

$$\widetilde{\mathbf{w}}^{\text{new}(j)} = \widetilde{\mathbf{w}}^{\text{old}(j)} - \widetilde{\mathbf{U}}_j \lambda_j.$$

Substituting into the constraint yields

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{U}}_j \lambda_j = \widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{old}(j)} - \mathbf{b}_j.$$

If the Gram matrix

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{U}}_j$$

is invertible, one obtains

$$\lambda_j = \left(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{U}}_j \right)^{-1} \left(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{old}(j)} - \mathbf{b}_j \right).$$

The least-change correction therefore becomes

$$\widetilde{\mathbf{w}}^{\text{new}(j)} = \widetilde{\mathbf{w}}^{\text{old}(j)} - \widetilde{\mathbf{U}}_j \left(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{U}}_j \right)^{-1} \left(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{old}(j)} - \mathbf{b}_j \right).$$

This expression shows that the update is an orthogonal projection of the previous weights onto the affine subspace defined by the reduced exactness constraints.

13.4 Interpretation as a minimum redistribution

The no-enforcement stage does not attempt to redesign the local cubature rule. Instead, it redistributes as little as possible the weights already present in the current feasible rule.

Suppose that point p carries only a small contribution in the current local weights. Then the correction required to absorb its removal is also expected to be small. In such cases, the redistribution remains localized and smooth. This is precisely the regime in which the no-enforcement stage is most effective.

Conversely, when the remaining support is already too sparse, the elimination of one point may require large compensatory redistributions. In that regime, the unconstrained least-change update may generate:

$$\text{negative weights,} \quad \text{large oscillations,} \quad \text{loss of feasibility.}$$

At that point, the algorithm transitions to the explicit positivity stage.

13.5 Why smoothness is usually better in this stage

The absence of active-set clamping is one of the main reasons why the no-enforcement stage tends to preserve smoothness.

At this stage, the adaptive weights are obtained as solutions of smooth linear least-change systems. Therefore, as long as the local operators vary smoothly with the manifold coordinate, the resulting weight field

$$\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$$

also tends to vary smoothly.

By contrast, once positivity is enforced, the feasible set becomes polyhedral. As the latent coordinate changes, different constraints may become active or inactive. These active-set transitions introduce nonsmoothness into the solution map.

Thus, even though the no-enforcement stage allows negative weights, it often produces more regular adaptive weight fields than the positivity-enforcing stages.

13.6 Acceptance and rejection of candidates

The unconstrained redistribution associated with a candidate point p is not accepted automatically. The updated weights must satisfy several practical criteria.

Typical checks include:

- satisfaction of the local exactness systems;
- absence of excessive negative weights;
- bounded redistribution magnitude;
- acceptable numerical conditioning.

If the candidate satisfies the prescribed tolerances, the elimination is committed:

$$\mathcal{I}_{\text{act}} \leftarrow \mathcal{I}_{\text{act}} \setminus \{p\},$$

and the updated adaptive weights become the new current solution:

$$\mathbf{W}^{\text{old}} \leftarrow \mathbf{W}^{\text{new}}.$$

Otherwise, the candidate is rejected and the next point in the ordered list is tested.

The important point is that rejection does not necessarily mean that the point is fundamentally indispensable. It only means that the current unconstrained redistribution stage cannot remove it while satisfying the prescribed criteria. The same point may later become removable once positivity enforcement or graph-regularized redistribution is activated.

13.7 Pseudo-algorithm

The no-enforcement redistribution stage can be summarized as follows.

Algorithm 4 No-enforcement least-change pruning

Require: Current support $\mathcal{I}_{\text{act}}$; adaptive weights $\mathbf{W}^{\text{old}}$; local systems $\{(\mathbf{U}_j, \mathbf{b}_j)\}_{j=1}^{N_M}$.

Ensure: Updated support and adaptive weights if a feasible elimination is found.

1: Compute aggregate weights

$$R_i = \sum_{j=1}^{N_M} W_{ij}.$$

2: Sort candidates in increasing order of R_i .

3: **for** candidate point p in the ordered list **do**

4: Define

$$\mathcal{I}_{\text{rem}} = \mathcal{I}_{\text{act}} \setminus \{p\}.$$

5: **for** $j = 1, \dots, N_M$ **do**

6: Restrict the local basis:

$$\widetilde{\mathbf{U}}_j = \mathbf{U}_j(\mathcal{I}_{\text{rem}}, :).$$

7: Compute the least-change redistribution:

$$\widetilde{\mathbf{w}}^{\text{new}(j)} = \widetilde{\mathbf{w}}^{\text{old}(j)} - \widetilde{\mathbf{U}}_j(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{U}}_j)^{-1}(\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{old}(j)} - \mathbf{b}_j).$$

8: Reconstruct the full local weights with

$$w_{p,\text{new}}^{(j)} = 0.$$

9: **end for**

10: Assemble the candidate adaptive weight matrix $\mathbf{W}^{\text{new}}$.

11: **if** $\mathbf{W}^{\text{new}}$ satisfies the acceptance criteria **then**

12: Commit the elimination:

$$\mathcal{I}_{\text{act}} \leftarrow \mathcal{I}_{\text{rem}}.$$

13: Update

$$\mathbf{W}^{\text{old}} \leftarrow \mathbf{W}^{\text{new}}.$$

14: Exit successfully.

15: **end if**

16: **end for**

17: Return failure if no candidate is accepted.

13.8 Function-level interpretation

The no-enforcement stage is implemented in

Loop_MAWecmNOENF_cl.m.

The suffix “NOENF” refers to the absence of explicit positivity enforcement. The suffix “cl” refers historically to the least-change redistribution logic.

The function operates on all local systems simultaneously, because the support must remain common across the manifold. However, the redistribution itself is performed independently at each manifold node. Therefore, the stage is local in the manifold direction and does not yet use the graph Laplacian.

Graph coupling enters only in the later graph-regularized active-set stage, which is described in the next sections.

14 Stage 2: local positivity enforcement by active sets

The no-enforcement redistribution stage described previously is often capable of removing a substantial number of integration points while preserving a relatively smooth adaptive weight field. However, as the support becomes progressively smaller, the unconstrained least-change redistribution may produce negative weights or large oscillatory corrections. At that point, explicit positivity enforcement becomes necessary.

The second redistribution stage of MAW-ECM therefore introduces nonnegativity constraints into the local redistribution problem. This stage is implemented in

`Loop_MAWecmNOENFe.m.`

Unlike the graph-based stage described later, the present formulation still treats each manifold node independently. The coupling between neighboring latent states is not yet introduced.

Thus, the present stage should be viewed as:

local positivity enforcement without graph coupling.

14.1 Local constrained redistribution problem

Suppose that a candidate point p is tentatively removed from the support. As before, define

$$\mathcal{I}_{\text{rem}} = \mathcal{I}_{\text{act}} \setminus \{p\},$$

and let

$$\widetilde{\mathbf{U}}_j = \mathbf{U}_j(\mathcal{I}_{\text{rem}}, :)$$

be the reduced local basis after elimination of the candidate point.

At manifold node j , the redistributed weights must satisfy:

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j.$$

In contrast with the no-enforcement stage, one now additionally imposes

$$\widetilde{\mathbf{w}}^{\text{new}(j)} \geq 0.$$

The redistribution problem therefore becomes

$$\min_{\widetilde{\mathbf{w}}^{\text{new}(j)}} \frac{1}{2} \|\widetilde{\mathbf{w}}^{\text{new}(j)} - \widetilde{\mathbf{w}}^{\text{old}(j)}\|_2^2,$$

subject to

$$\widetilde{\mathbf{U}}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j,$$

and

$$\widetilde{\mathbf{w}}^{\text{new}(j)} \geq 0.$$

Thus, the local adaptive weights are redistributed as little as possible while remaining both feasible and nonnegative.

14.2 Geometric interpretation

Without positivity enforcement, the feasible set of the redistribution problem is an affine subspace:

$$\mathcal{A}_j = \left\{ \mathbf{w} : \widetilde{\mathbf{U}}_j^T \mathbf{w} = \mathbf{b}_j \right\}.$$

The no-enforcement redistribution is simply the orthogonal projection of the previous weights onto this affine subspace.

With positivity enforcement, the feasible set becomes

$$\mathcal{F}_j = \left\{ \mathbf{w} : \widetilde{\mathbf{U}}_j^T \mathbf{w} = \mathbf{b}_j, \quad \mathbf{w} \geq 0 \right\}.$$

This set is no longer an affine space, but a convex polyhedron obtained by intersecting the affine exactness constraints with the positive orthant.

The redistribution problem therefore becomes a constrained projection onto a polyhedral feasible set.

14.3 Active-set philosophy

The implementation adopts an active-set strategy. The idea is simple: whenever one component of the redistributed weights becomes negative, that component is clamped to zero and treated as an additional equality constraint.

At manifold node j , define the local active set

$$\mathcal{D}_j \subset \{1, \dots, |\mathcal{I}_{\text{rem}}|\}.$$

An index

$$i \in \mathcal{D}_j$$

means that the corresponding local weight is fixed to zero:

$$w_i^{(j)} = 0.$$

The remaining local weights are free variables.

Thus, the constrained redistribution problem is converted into a sequence of equality-constrained least-change problems with progressively larger active sets.

14.4 Reduced local variables

Let

$$\mathcal{I}_{\text{free}j} = \{1, \dots, |\mathcal{I}_{\text{rem}}|\} \setminus \mathcal{D}_j$$

denote the free local indices at manifold node j . Restricting the local basis to the free variables gives

$$\widehat{\mathbf{U}}_j = \widetilde{\mathbf{U}}_j(\mathcal{I}_{\text{free}j}, :).$$

Similarly, define the free local unknown

$$\widehat{\mathbf{w}}^{\text{new}(j)} = \widetilde{\mathbf{w}}^{\text{new}(j)}(\mathcal{I}_{\text{free}j}).$$

The local constrained redistribution problem then becomes

$$\min_{\widehat{\mathbf{w}}^{\text{new}(j)}} \frac{1}{2} \|\widehat{\mathbf{w}}^{\text{new}(j)} - \widehat{\mathbf{w}}^{\text{old}(j)}\|_2^2,$$

subject to

$$\widehat{\mathbf{U}}_j^T \widehat{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j.$$

This has exactly the same algebraic structure as the no-enforcement stage, but restricted to the free variables.

14.5 Iterative evolution of the active sets

The active-set redistribution proceeds iteratively.

Starting from

$$\mathcal{D}_j = \emptyset,$$

one computes the least-change redistribution over all remaining variables. If all components are nonnegative, the candidate elimination is accepted.

Otherwise, one identifies the negative components:

$$\mathcal{N}_j = \left\{ i : w_i^{(j)} < 0 \right\}.$$

These indices are added to the active set:

$$\mathcal{D}_j \leftarrow \mathcal{D}_j \cup \mathcal{N}_j.$$

The redistribution problem is then solved again over the reduced free set.

This process is repeated until:

- all free variables are nonnegative, or
- the free system becomes infeasible or singular.

Thus, the active sets evolve adaptively during the redistribution process.

14.6 Node-dependent active sets

A crucial aspect of the present implementation is that the active sets are local to each manifold node.

Suppose that integration-point weight i becomes negative at manifold node j_1 . Then

$$i \in \mathcal{D}_{j_1},$$

but it does not follow that

$$i \in \mathcal{D}_{j_2}$$

for another node j_2 .

Thus, positivity enforcement is local in latent space:

$$\mathcal{D}_{j_1} \neq \mathcal{D}_{j_2} \quad \text{in general.}$$

This distinction is important. The support itself remains common over the manifold, but the local positivity constraints do not.

Consequently, even after the support has been fixed, the local constrained redistribution problem may have different active variables at different latent states.

This is one of the main mechanisms generating nonsmoothness in the adaptive weight field.

14.7 Origin of nonsmoothness

At the no-enforcement stage, the redistribution is obtained from smooth linear systems. Therefore, the resulting mapping

$$\mathbf{q}_M \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_M)$$

tends to vary smoothly.

Once positivity is enforced, however, the active sets may change abruptly as the latent coordinate varies. The adaptive weight field therefore becomes the solution map of a piecewise-defined constrained optimization problem.

Whenever the active set changes, the local affine system defining the redistribution also changes. As a consequence:

the adaptive weight field is generally only piecewise smooth.

This effect becomes increasingly pronounced as the support becomes more compressed.

The graph-based stage introduced later is intended precisely to moderate this behavior by coupling neighboring latent states during the redistribution.

14.8 Acceptance and rejection

A candidate point p is accepted only if the positivity-enforcing active-set procedure succeeds at every manifold node.

More precisely, for all

$$j = 1, \dots, N_M,$$

the final redistributed weights must satisfy:

$$\widetilde{U}_j^T \widetilde{\mathbf{w}}^{\text{new}(j)} = \mathbf{b}_j,$$

and

$$\widetilde{\mathbf{w}}^{\text{new}(j)} \geq 0,$$

up to the prescribed tolerances.

If the constrained redistribution fails at even one manifold node, the candidate elimination is rejected.

Thus, feasibility is global with respect to the sampled manifold, even though the redistributions themselves are computed locally.

14.9 Pseudo-algorithm

The local active-set redistribution can be summarized as follows.

Algorithm 5 Local positivity-enforcing redistribution

Require: Candidate point p ; current adaptive weights $\mathbf{W}^{\text{old}}$; local systems $\{(\mathbf{U}_j, \mathbf{b}_j)\}_{j=1}^{N_M}$.

Ensure: Updated adaptive weights if the candidate is feasible.

1: Define

$$\mathcal{I}_{\text{rem}} = \mathcal{I}_{\text{act}} \setminus \{p\}.$$

2: **for** $j = 1, \dots, N_M$ **do**

3: Initialize

$$\mathcal{D}_j = \emptyset.$$

4: **while** active-set iteration has not converged **do**

5: Define the free local indices

$$\mathcal{I}_{\text{free}j} = \{1, \dots, |\mathcal{I}_{\text{rem}}|\} \setminus \mathcal{D}_j.$$

6: Restrict the basis to the free variables

$$\widehat{\mathbf{U}}_j = \widetilde{\mathbf{U}}_j(\mathcal{I}_{\text{free}j}, :).$$

7: Solve the free least-change redistribution problem.

8: **if** all free variables are nonnegative **then**

9: Accept the local redistribution.

10: Exit the local active-set loop.

11: **else**

12: Detect negative free variables.

13: Add them to the active set:

$$\mathcal{D}_j \leftarrow \mathcal{D}_j \cup \mathcal{N}_j.$$

14: **end if**

15: **end while**

16: **if** the local redistribution failed **then**

17: Reject the candidate point p globally.

18: Exit with failure.

19: **end if**

20: **end for**

21: Accept the candidate point p globally.

14.10 Interpretation

The local active-set stage can be viewed as a compromise between two competing requirements.

On one hand, the adaptive weights should remain close to the previous feasible solution:

$$\mathbf{W}^{\text{new}} \approx \mathbf{W}^{\text{old}}.$$

On the other hand, positivity must be enforced:

$$\mathbf{W}^{\text{new}} \geq 0.$$

As the support becomes smaller, these two requirements become increasingly difficult to reconcile simultaneously.

The resulting constrained redistribution is therefore intrinsically nonlinear and nonsmooth. The graph-based stage described next attempts to compensate for this by coupling neighboring manifold states and discouraging abrupt local redistributions.

14.11 Function-level interpretation

The local positivity-enforcing stage is implemented in

`Loop_MAWecmNOENFe.m`.

The suffix “NOENFe” historically refers to the no-overlap local active-set version with explicit enforcement.

The key structural property of this routine is:

local active sets, but common support.

The support elimination is global, but the positivity constraints are enforced independently at each manifold node.

This distinction will become important in the graph-based stage, where the redistribution is instead performed through a coupled optimization problem over all manifold states simultaneously.

15 Stage 3: graph-regularized positivity enforcement

The local active-set stage enforces positivity, but it treats each manifold sample independently. This independence is convenient algebraically, yet it may produce weight fields with sharp variations across neighboring latent states. The graph-regularized stage is introduced to reduce this effect.

This stage is implemented through

`GraphBased_MAW_ECM_2v.m`,

which calls the core redistribution routine

`prune_step_optionB.m`.

The essential change with respect to the local active-set stage is that the weights at all manifold samples are updated simultaneously. The graph Laplacian $\mathbf{K}_G$ couples neighboring latent nodes and penalizes rough redistributions.

15.1 Global unknown and constraints

Let the current adaptive weights be

$$\mathbf{W}^{\text{old}} \in \mathbb{R}^{r \times N_M},$$

where r denotes the number of currently retained candidate points at this stage of the pruning loop. A tentative elimination of point p seeks updated weights

$$\mathbf{W}^{\text{new}} \in \mathbb{R}^{r \times N_M}.$$

The pruning condition is imposed globally:

$$W_{pj}^{\text{new}} = 0, \quad j = 1, \dots, N_M.$$

For each manifold node j , the local exactness condition remains

$$A_j \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j,$$

where, in the notation of the implementation,

$$A_j = \mathbf{U}_j^T$$

restricted to the rows currently active in the pruning trial.

The positivity condition is

$$\mathbf{w}^{\text{new}(j)} \geq 0, \quad j = 1, \dots, N_M.$$

15.2 Incremental graph-regularized objective

The graph-based update minimizes a least-change functional augmented with a smoothness penalty over the latent graph. In incremental form, the objective is

$$\frac{1}{2} \|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2 + \frac{\alpha_G}{2} \text{tr} \left[(\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}) \mathbf{K}_G (\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}})^T \right].$$

The first term keeps the new adaptive weights close to the previous feasible weights. The second term penalizes rough variations of the update over the sampled latent manifold.

The use of the increment

$$\Delta \mathbf{W} = \mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}$$

is important. It means that the graph regularization acts on the redistribution caused by the pruning step, not necessarily on the absolute value of the weights. Thus, a weight field that is already nonsmooth is not directly forced to become smooth in one step; rather, the new pruning-induced change is discouraged from introducing additional sharp variations.

If the absolute version is used instead, the graph term is applied directly to $\mathbf{W}^{\text{new}}$. The incremental version is usually less invasive and better aligned with the idea of progressive pruning.

15.3 Stacked quadratic form

The implementation solves the graph-regularized redistribution in stacked-vector form. Define

$$\mathbf{z} = \text{vec}(\mathbf{W}^{\text{new}}), \quad \mathbf{z}^{\text{old}} = \text{vec}(\mathbf{W}^{\text{old}}),$$

with columnwise stacking. Then

$$\|\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}\|_F^2 = \|\mathbf{z} - \mathbf{z}^{\text{old}}\|_2^2,$$

and

$$\text{tr} \left[(\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}}) \mathbf{K}_G (\mathbf{W}^{\text{new}} - \mathbf{W}^{\text{old}})^T \right] = (\mathbf{z} - \mathbf{z}^{\text{old}})^T (\mathbf{K}_G \otimes \mathbf{I}_r) (\mathbf{z} - \mathbf{z}^{\text{old}}).$$

Therefore, the objective can be written as

$$\frac{1}{2} (\mathbf{z} - \mathbf{z}^{\text{old}})^T \mathbf{H}_G (\mathbf{z} - \mathbf{z}^{\text{old}}),$$

where

$$\mathbf{H}_G = \mathbf{I} + \alpha_G (\mathbf{K}_G \otimes \mathbf{I}_r).$$

Expanding the quadratic form and dropping constants gives

$$\frac{1}{2} \mathbf{z}^T \mathbf{H}_G \mathbf{z} - \mathbf{g}^T \mathbf{z},$$

with

$$\mathbf{g} = \mathbf{H}_G \mathbf{z}^{\text{old}}$$

in the incremental formulation.

The matrix $\mathbf{H}_G$ is symmetric positive definite for $\alpha_G \geq 0$, since $\mathbf{K}_G$ is positive semidefinite and the identity term is present. This is important for the robustness of the reduced nullspace solve.

15.4 Node-dependent active sets

The graph-based implementation enforces nonnegativity through node-dependent active sets. For each manifold node j , define

$$\mathcal{D}_j \subset \{1, \dots, r\}.$$

The interpretation is:

$$i \in \mathcal{D}_j \iff W_{ij}^{\text{new}} = 0.$$

The pruning candidate is clamped at all nodes:

$$p \in \mathcal{D}_j, \quad j = 1, \dots, N_M.$$

Additional indices are added to $\mathcal{D}_j$ only at the nodes where negative weights are detected.

This is the key distinction of the option implemented in `prune_step_optionB.m`: a weight can be clamped at one manifold node without being clamped at all other nodes. Hence, the active set is local in the latent direction, even though the update is graph-coupled.

15.5 Nullspace parametrization

For a fixed collection of active sets

$$\{\mathcal{D}_j\}_{j=1}^{N_M},$$

the problem contains only equality constraints: local exactness constraints and clamping constraints. The implementation solves this equality-constrained quadratic problem by nullspace parametrization.

Let

$$\mathbf{z} = \mathbf{z}_p + \mathbf{N}\mathbf{y},$$

where:

- $\mathbf{z}_p$ is one particular vector satisfying all equality constraints;
- $\mathbf{N}$ is a basis for the nullspace of the equality-constraint matrix;
- $\mathbf{y}$ contains the reduced unknowns.

Substitution into the expanded quadratic objective gives the reduced system

$$\mathbf{H}_{\text{red}}\mathbf{y} = \mathbf{f}_{\text{red}},$$

where

$$\mathbf{H}_{\text{red}} = \mathbf{N}^T \mathbf{H}_G \mathbf{N},$$

and

$$\mathbf{f}_{\text{red}} = \mathbf{N}^T (\mathbf{g} - \mathbf{H}_G \mathbf{z}_p).$$

Since $\mathbf{H}_G$ is positive definite and $\mathbf{N}$ has full column rank, the matrix $\mathbf{H}_{\text{red}}$ is also positive definite. The reduced system can therefore be solved directly.

The full updated weight vector is then reconstructed from

$$\mathbf{z} = \mathbf{z}_p + \mathbf{N}\mathbf{y},$$

and reshaped into

$$\mathbf{W}^{\text{new}} \in \mathbb{R}^{r \times N_M}.$$

15.6 Active-set iterations

After solving the equality-constrained graph-regularized problem for the current active sets, the algorithm checks nonnegativity. For each node j , the free variables are inspected. If

$$W_{ij}^{\text{new}} < -\varepsilon_{\text{neg}},$$

for some free index i , that index is added to the local active set:

$$\mathcal{D}_j \leftarrow \mathcal{D}_j \cup \{i\}.$$

The graph-coupled problem is then solved again with the enlarged active sets.

The active-set iteration stops when no new negative weights are detected. If the maximum number of active-set iterations is reached, or if the constrained system becomes infeasible, the pruning candidate is rejected.

Thus, one graph-based pruning trial consists of:

fix candidate $p \rightarrow$ solve graph-coupled equality problem $\rightarrow$ update active sets $\rightarrow$ repeat until feasible or rejected

15.7 Why graph coupling helps

In the local active-set stage, each manifold node redistributes its weights independently. If a weight is clamped at one node but not at its neighbors, the resulting field may develop a localized jump. The graph-based stage reduces this effect by penalizing large differences of the update across neighboring latent nodes.

For an edge $(a, b) \in \mathcal{E}_M$, the update jump associated with integration point i is

$$\Delta W_{ia} - \Delta W_{ib}.$$

The graph energy penalizes such jumps in an averaged quadratic sense. Therefore, among the feasible redistributions, the graph-based update favors those with more coherent changes across the latent manifold.

This does not eliminate active-set nonsmoothness. If the constraints and positivity conditions force a local clamp, the graph penalty cannot remove it. It can only choose, among the feasible solutions, the one that is least rough according to the chosen graph metric.

15.8 Pseudo-algorithm

Algorithm 6 Graph-regularized active-set redistribution for one pruning candidate

Require: Current weights $\mathbf{W}^{\text{old}}$; local systems $\{(A_j, \mathbf{b}_j)\}_{j=1}^{N_M}$; graph Laplacian $\mathbf{K}_G$; smoothing parameter α_G ; candidate point p .

Ensure: Updated weights $\mathbf{W}^{\text{new}}$ if the pruning candidate is feasible.

1: Initialize active sets:

$$\mathcal{D}_j \leftarrow \{p\}, \quad j = 1, \dots, N_M.$$

2: Build

$$\mathbf{H}_G = \mathbf{I} + \alpha_G(\mathbf{K}_G \otimes \mathbf{I}_r).$$

3: **if** incremental smoothing is used **then**

4: Set

$$\mathbf{g} = \mathbf{H}_G \mathbf{z}^{\text{old}}.$$

5: **else**

6: Set

$$\mathbf{g} = \mathbf{z}^{\text{old}}.$$

7: **end if**

8: **for** active-set iteration $k = 1, \dots, k_{\text{max}}$ **do**

9: Build the global equality constraints from:

$$A_j \mathbf{w}^{\text{new}(j)} = \mathbf{b}_j, \quad W_{ij}^{\text{new}} = 0 \quad \text{for } i \in \mathcal{D}_j.$$

10: Compute a particular solution $\mathbf{z}_p$ and nullspace basis $\mathbf{N}$.

11: Solve

$$(\mathbf{N}^T \mathbf{H}_G \mathbf{N}) \mathbf{y} = \mathbf{N}^T (\mathbf{g} - \mathbf{H}_G \mathbf{z}_p).$$

12: Reconstruct

$$\mathbf{z} = \mathbf{z}_p + \mathbf{N} \mathbf{y}, \quad \mathbf{W}^{\text{new}} = \text{reshape}(\mathbf{z}).$$

13: Detect negative free weights.

14: **if** no new negative weights are detected **then**

15: Accept the candidate and return success.

16: **else**

17: Add the negative entries to the corresponding node-dependent active sets $\mathcal{D}_j$.

18: **end if**

19: **end for**

20: Reject the candidate if convergence is not achieved.

15.9 Function-level interpretation

At the orchestration level, graph-based pruning is performed by

`GraphBased_MAW_ECM_2v.m.`

This routine tests pruning candidates, calls the graph-regularized update, and selects one candidate to commit.

The actual graph-regularized active-set solve for one candidate is implemented in

`prune_step_optionB.m.`

This routine receives the current weights, the local exactness systems, the graph Laplacian, the smoothing parameter, and the candidate index. It returns the updated weights and diagnostic information indicating whether the candidate was feasible.

The name “Option B” refers to the node-dependent active-set strategy: negative weights are clamped only at the manifold nodes where they occur, rather than globally across all nodes.

16 Regression of the adaptive weight fields

After the pruning stage has terminated, MAW-ECM provides:

$$\mathcal{Z}_{\text{red}} \subset \mathcal{Z}_{\text{FE}},$$

together with the sampled adaptive weights

$$\mathbf{W}_{\text{red}} \in \mathbb{R}^{n_{\text{red}} \times N_{\text{M}}}.$$

The columns of $\mathbf{W}_{\text{red}}$ correspond to the sampled latent coordinates

$$\{\mathbf{q}_{\text{M}}^{(j)}\}_{j=1}^{N_{\text{M}}}.$$

At this stage, the adaptive weights are known only at the sampled manifold states used during the offline stage. An additional approximation step is therefore required in order to evaluate the adaptive cubature rule at new online latent coordinates.

The purpose of the present stage is to construct the mapping

$$\mathbf{q}_{\text{M}} \longmapsto \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}).$$

This stage is implemented in

`RegressionViaRBF_master_weights.m.`

16.1 Weight fields over the latent manifold

Each retained integration point defines one scalar field over the latent manifold. Let

$$\mathbf{w}_i = [W_{i1} \quad W_{i2} \quad \cdots \quad W_{iN_{\text{M}}}]^T \in \mathbb{R}^{N_{\text{M}}}$$

collect the adaptive weights associated with retained integration point i over all sampled manifold states.

Thus, the adaptive cubature rule is not represented by a single vector of weights, but by a family of scalar fields:

$$\{w_i(\mathbf{q}_{\text{M}})\}_{i=1}^{n_{\text{red}}}.$$

The regression stage approximates these scalar fields from the sampled data

$$\left\{ \mathbf{q}_{\text{M}}^{(j)}, W_{ij} \right\}_{j=1}^{N_{\text{M}}}.$$

16.2 Why regression is necessary

The pruning stage operates only on the sampled manifold states available during offline training. However, during online evaluation, the latent coordinates will generally not coincide with the sampled nodes:

$$\mathbf{q}_{\text{Monline}} \notin \{\mathbf{q}_{\text{M}}^{(j)}\}_{j=1}^{N_{\text{M}}}.$$

Therefore, the adaptive weights must be interpolated or approximated.

The online adaptive cubature rule takes the form

$$I(\mathbf{q}_{\text{Monline}}) \approx \sum_{i \in \mathcal{Z}_{\text{red}}} f_i(\mathbf{q}_{\text{Monline}}) w_i(\mathbf{q}_{\text{Monline}}).$$

Hence, online evaluation requires the ability to compute

$$\mathbf{w}_{\text{red}}(\mathbf{q}_{\text{Monline}}).$$

The regression stage should therefore be interpreted as the final step that converts the sampled adaptive weights into a continuous surrogate model over the latent manifold.

16.3 Choice of radial basis functions

The present implementation uses radial basis functions (RBFs). This choice is motivated by several considerations.

First, the latent dimension is very small:

$$r_M = 2.$$

Second, the sampled latent manifold is structured and relatively regular. Third, the adaptive weights, although not perfectly smooth, usually vary in a coherent manner over the manifold after graph regularization.

Under these conditions, RBF interpolation provides:

- meshfree approximation,
- smooth evaluation,
- simple implementation,
- differentiability of the resulting surrogate,
- flexibility with respect to irregular sampling.

The current implementation uses anisotropic Gaussian kernels.

16.4 RBF approximation of one weight field

Consider one retained integration point i . The corresponding scalar weight field is approximated as

$$w_i(\mathbf{q}_M) \approx \sum_{k=1}^{n_{\text{RBF}}} \gamma_{ik} \phi_k(\mathbf{q}_M) + p_i(\mathbf{q}_M),$$

where:

- ϕ_k are radial basis functions centered at selected latent samples,
- γ_{ik} are the regression coefficients,
- $p_i(\mathbf{q}_M)$ is a low-order polynomial correction.

For Gaussian kernels,

$$\phi_k(\mathbf{q}_M) = \exp \left(-\frac{1}{2} (\mathbf{q}_M - \mathbf{c}_k)^T \mathbf{D}_k^{-2} (\mathbf{q}_M - \mathbf{c}_k) \right),$$

where:

- $\mathbf{c}_k$ is the center of the k -th RBF,
- $\mathbf{D}_k$ defines anisotropic characteristic lengths.

The polynomial correction may include:

$$p_i(\mathbf{q}_M) = a_i, \quad \text{or} \quad p_i(\mathbf{q}_M) = a_i + \mathbf{b}_i^T \mathbf{q}_M,$$

depending on the selected approximation order.

16.5 Structured subsampling of RBF centers

Using all sampled manifold states as RBF centers is often unnecessary and may lead to large dense interpolation systems. The implementation therefore performs structured subsampling of the latent grid.

Let

$$\{\mathbf{c}_k\}_{k=1}^{n_{\text{RBF}}}$$

denote the retained centers. Typically,

$$n_{\text{RBF}} \ll N_{\text{M}}.$$

The centers are selected so as to preserve coverage of the latent domain while reducing redundancy among nearby samples.

This choice is especially useful because neighboring latent states usually carry very similar adaptive weights after graph regularization.

16.6 Matrix form of the regression problem

For one integration point i , define the sampled values

$$\mathbf{w}_i = \begin{bmatrix} W_{i1} \\ \vdots \\ W_{iN_{\text{M}}} \end{bmatrix}.$$

Define also the interpolation matrix

$$\Phi \in \mathbb{R}^{N_{\text{M}} \times n_{\text{RBF}}},$$

with entries

$$\Phi_{jk} = \phi_k(\mathbf{q}_{\text{M}}^{(j)}).$$

If polynomial correction is included, let

$$\mathbf{P}$$

denote the polynomial basis evaluated at the sampled latent coordinates.

The regression problem can then be written schematically as

$$\begin{bmatrix} \Phi & \mathbf{P} \end{bmatrix} \begin{bmatrix} \gamma_i \\ \boldsymbol{\eta}_i \end{bmatrix} \approx \mathbf{w}_i.$$

Depending on the selected options, the implementation may use:

- interpolation,
- least-squares approximation,
- regularized least squares.

The regression is performed independently for each retained integration point.

16.7 Differentiability

One important consequence of the RBF representation is that the adaptive weights become differentiable functions of the latent coordinates:

$$\mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}) \in C^\infty$$

for Gaussian kernels.

Thus, one may compute:

$$\frac{\partial \mathbf{w}_{\text{red}}}{\partial \mathbf{q}_{\text{M}}}, \quad \frac{\partial^2 \mathbf{w}_{\text{red}}}{\partial \mathbf{q}_{\text{M}}^2}.$$

This property may become useful in future extensions involving:

- tangent-consistent hyperreduction,
- latent-space continuation,
- adaptive online integration,
- reduced Newton schemes,
- sensitivity analysis.

It should be emphasized, however, that the smoothness of the RBF surrogate does not imply that the underlying sampled weights are themselves smooth. If the offline adaptive weights contain strong active-set transitions, the RBF approximation simply produces a smooth interpolation of a piecewise-smooth dataset.

16.8 Influence of graph regularization

The graph-regularized pruning stage plays an important role in the quality of the regression problem.

Without graph regularization, neighboring manifold samples may carry strongly different adaptive weights due to local active-set changes. In that case, the mapping

$$\mathbf{q}_{\text{M}} \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}})$$

becomes more difficult to regress accurately.

The graph penalty mitigates this effect by discouraging abrupt redistribution patterns across neighboring latent states. Consequently:

- the sampled adaptive weights become more coherent,
- the regression problem becomes better conditioned,
- fewer RBF centers are generally required,
- online interpolation becomes more stable.

Thus, graph regularization should be viewed not only as a stabilization device for pruning, but also as a preprocessing step for the final surrogate model of the adaptive weights.

16.9 Online evaluation

Once the regression coefficients have been computed, online evaluation proceeds as follows.

Given a latent coordinate

$$\mathbf{q}_{\text{Monline}},$$

the RBF surrogate evaluates

$$\mathbf{w}_{\text{red}}(\mathbf{q}_{\text{Monline}}) \in \mathbb{R}^{n_{\text{red}}}.$$

The online adaptive cubature approximation is then

$$I(\mathbf{q}_{\text{Monline}}) \approx \sum_{i \in \mathcal{Z}_{\text{red}}} f_i(\mathbf{q}_{\text{Monline}}) w_i(\mathbf{q}_{\text{Monline}}).$$

Thus, online evaluation requires:

- evaluation of the reduced set of integrands,
- evaluation of the adaptive weights through the RBF surrogate.

Importantly, the online stage does not solve a new ECM problem. All adaptive behavior has already been encoded offline in the learned mapping

$$\mathbf{q}_{\text{M}} \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}).$$

16.10 Pseudo-algorithm

Algorithm 7 Regression of the adaptive weight fields

Require: Sampled latent coordinates

$$\{\mathbf{q}_{\text{M}}^{(j)}\}_{j=1}^{N_{\text{M}}};$$

sampled adaptive weights

$$\mathbf{W}_{\text{red}};$$

RBF parameters and regression options.

Ensure: Surrogate model

$$\mathbf{q}_{\text{M}} \mapsto \mathbf{w}_{\text{red}}(\mathbf{q}_{\text{M}}).$$

1: Select the RBF centers

$$\{\mathbf{c}_k\}_{k=1}^{n_{\text{RBF}}}.$$

2: Build the interpolation matrix

$$\Phi_{jk} = \phi_k(\mathbf{q}_{\text{M}}^{(j)}).$$

3: **for** each retained integration point $i = 1, \dots, n_{\text{red}}$ **do**

4: Extract the sampled weight field

$$\mathbf{w}_i = \begin{bmatrix} W_{i1} \\ \vdots \\ W_{iN_{\text{M}}} \end{bmatrix}.$$

5: Solve the RBF regression problem.

6: Store the coefficients:

$$\gamma_i, \quad \boldsymbol{\eta}_i.$$

7: **end for**

8: Return the RBF surrogate model.

16.11 Function-level interpretation

The regression stage is implemented in

`RegressionViaRBF_master_weights.m.`

This routine receives:

- the sampled latent coordinates,
- the adaptive weights,
- the RBF interpolation parameters.

The resulting regression structure is then evaluated online through routines of the same family as:

`EvalSlaveRBF_Online.m,`

although specialized to the adaptive weight fields.

The present implementation is specifically tuned for:

$$r_M = 2,$$

structured latent grids, and visualization of the adaptive weight surfaces. However, the regression formulation itself is not restricted to the bivariate case. The same approximation strategy extends naturally to higher-dimensional latent manifolds, provided that:

- the latent coordinates remain sufficiently low-dimensional,
- the sampled manifold is adequately covered,
- the adaptive weights vary coherently over the latent space.

17 A physical interpretation of the pruning process

Before continuing with the mathematical details of the redistribution and active-set procedures, it is useful to pause and develop a more physical and perhaps more appealing interpretation of the MAW–ECM mechanism.

The goal of the present section is not mathematical rigor. Rather, the idea is to help the reader build an intuitive mental picture of what the algorithm is actually doing. In practice, many aspects of the method become much easier to understand once one stops thinking in terms of matrices and optimization variables and instead imagines a physical structure evolving under multiple loading conditions.

The analogy that follows should therefore be understood mainly as a conceptual device intended to facilitate understanding of the method without entering immediately into all the subtleties of constrained optimization.

17.1 A suspended bridge under many wind conditions

Imagine a very large suspended bridge composed of a dense network of tensile cables. Suppose now that the bridge must remain in equilibrium not under one single loading condition, but under an entire family of nearby conditions: slightly different wind directions, slightly different traffic distributions, slightly different deformation states, and so on.

This is the role played by the manifold samples

$$\mathbf{q}_M^{(j)}.$$

Each manifold point may be interpreted as one loading configuration of the same bridge.

Now imagine that, initially, the bridge has been designed in a very conservative way. There are many more cables than are truly necessary. The structure is highly redundant.

However, not all cables play the same role.

Some cables are absolutely crucial. They carry large portions of the load and participate in balancing many equilibrium conditions simultaneously. Others are less important and contribute only marginally.

This is precisely what one observes in the initial ECM rule.

Empirically, the ECM weights are highly nonuniform: some weights become much larger than others. This is not accidental. It is a direct consequence of the greedy nature of ECM. The algorithm progressively selects those integration points that are most effective for reproducing the reduced equilibrium conditions.

Under the structural analogy, the ECM algorithm has already identified the principal load paths of the bridge.

The resulting structure is therefore not homogeneous. Some cables carry large tensions, while others play only secondary corrective roles.

17.2 The frozen-force philosophy of standard ECM

At this point, one may imagine the philosophy of a standard fixed-weight ECM rule in the following way.

Suppose the engineer designing the bridge imposes the following artificial restriction:

every cable must always carry exactly the same tension pattern for all loading configurations.

Immediately, the bridge becomes much harder to optimize.

Why?

Because different loading conditions naturally require different stress redistributions. A cable that is extremely important under one wind direction may become almost irrelevant under another.

Nevertheless, the fixed-weight ECM philosophy forces the same force distribution to remain valid everywhere on the manifold.

As a consequence, the bridge requires many more cables than are locally necessary under individual loading states.

This is precisely the main limitation of a fixed manifold ECM rule.

17.3 The central idea of MAW-ECM

The central idea of MAW-ECM is almost deceptively simple.

Instead of forcing the cable tensions to remain frozen over all loading conditions, one allows the internal force distribution to adapt to the loading configuration.

The bridge keeps the same surviving cables:

same support,

but the tensions carried by those cables may vary from one loading state to another:

adaptive weights.

Thus, the adaptive weights

$$\mathbf{w}^{(j)}$$

may be interpreted as the tensions carried by the surviving cables under the loading configuration associated with manifold point $\mathbf{q}_M^{(j)}$.

This immediately introduces additional flexibility into the system.

A cable that is critically important for one loading condition may carry very little tension under another. The structure can now reorganize its internal forces locally over the manifold.

17.4 Pruning as cutting cables

Now imagine beginning a process of progressive simplification.

One cable is selected and cut.

Immediately, the external loading conditions remain exactly the same. The wind does not disappear merely because one cable has been removed. Therefore, the remaining cables must reorganize their tensions so as to preserve equilibrium.

At the beginning, this is easy.

The bridge is highly redundant. There are many alternative equilibrium paths. When one cable disappears, neighboring cables simply increase their tensions slightly and the structure remains perfectly stable.

This is the meaning of the null space in the MAW–ECM formulation.

The null-space directions represent admissible internal force redistributions that preserve the same external equilibrium resultants. The structure can “wiggle” its internal stresses while keeping the same global balance.

However, every removed cable reduces the redundancy of the structure.

The remaining bridge becomes progressively more stressed:

- fewer cables remain;
- each cable carries more responsibility;
- the redistribution freedom decreases;
- the null space shrinks.

Eventually, the bridge approaches a critical regime in which the surviving cables are almost uniquely determined by equilibrium.

17.5 Slacking cables and active sets

Now comes the most important part of the analogy.

Suppose that, after removing another cable, the redistribution problem attempts to assign a negative tension to one of the surviving members.

Mechanically, this means:

the structure is asking a cable to push.

But a cable cannot push.

A cable can only pull.

This is precisely the meaning of a negative weight in MAW–ECM.

The positivity constraint

$$W_{ij} \geq 0$$

therefore expresses a tension-only admissibility condition.

The active-set mechanism then acquires a completely natural interpretation.

Whenever a cable attempts to enter compression, it simply goes slack:

$$W_{ij} = 0.$$

Importantly, this does not necessarily remove the cable globally from the bridge. The same cable may:

- carry tension under some loading configurations,
- become slack under others,
- and reactivate elsewhere on the manifold.

This distinction is fundamental.

Pruning corresponds to permanently cutting a cable from the bridge.

The active set, by contrast, determines which surviving cables are actually load-bearing under each local loading configuration.

17.6 Regularization as coherent structural behavior

Finally, the role of regularization also becomes very intuitive.

Neighboring manifold points correspond to nearby loading configurations. One therefore expects nearby loading conditions to produce nearby internal stress patterns.

Without regularization, however, tiny changes in the loading state may produce wildly different redistributions:

- one cable active here,
- slack nearby,
- active again elsewhere.

The structure would behave erratically.

The regularization discourages this behavior. It favors coherent stress evolution throughout the manifold and promotes smoother transitions between neighboring loading configurations.

17.7 Final picture

Under this interpretation, MAW–ECM may be viewed as a progressive search for the smallest admissible tensile skeleton capable of supporting an entire family of neighboring equilibrium states.

The initial ECM rule provides an already compressed bridge obtained through a greedy equilibrium process. The pruning stage progressively cuts cables from the structure. The remaining cables continuously reorganize their tensions so as to preserve equilibrium. The null space measures the remaining redundancy of the bridge. The active set describes locally slack cables. The regularization promotes coherent structural behavior over the manifold.

Although purely conceptual, this picture captures surprisingly well the true mechanism underlying the MAW–ECM redistribution process.